ASPiK (Audio Specific Plugin Kernel) is the simplest and fastest way to get your professional AU, AAX and VST plugins designed and coded for Windows and MacOS. You only need the target SDKs (all are public/free) and a DAW of your choice. You can design and develop your plugin and assemble the GUI right inside of your DAW while it is running. You can create a new blank plugin project in a few seconds using the ASPiKreator software. And if you are a RackAFX7 user, you can now export your RAFX2 plugins in ASPiK format thanks to an agreement with the author.

Quick Start Guide

- ASPiK Project Folder
- Setting up your SDKs
- Create a new Project using the ASPiKreator (wicked easy!)
- Optional: Create a new Project Manually (for those that want to)
- Running CMake
- AAX Dev Notes
- AU Dev Notes
- VST Dev Notes
- Write a Plugin
- Design a GUI
- Example Projects

The ASPiK Project Folder

VSTGUI4 Library

ASPiK uses the VSTGUI4 library for all GUI rendering and design (yep, you get a free drag-and-drop GUI designer built-in to your plugin projects). You will first need to get the VSTGUI4 library package. You can download it with the latest VST3 SDK or from GitHub. ASPiK tries to keep up with the latest releasses of all SDKs. For the VSTGUI4 package, we ALWAYS follow the VST3 SDK release as VSTGUI releases are timed to coincide with VST3 SDK releases (note that the two products are different and have different licensing rules).

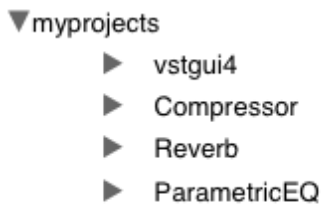
The latest version of VSTGUI4 is included in the ASPiK SDK in its entirety. You may use it to paste into your projects folder or you may substitute newer versions (check the latest requirements at http://www.aspikplugins.com**).**

The entire VSTGUI4 library is contained in an outer folder named simply "vstgui4" and within that folder is an inner folder named "vstgui." You need to have the outermost vstgui4 folder available to paste into your ASPiK projects folder. All plugin projects (for a given API or for the Universal API paradigm) require the presence of the vstgui4 subfolder regardless of whether they expose a

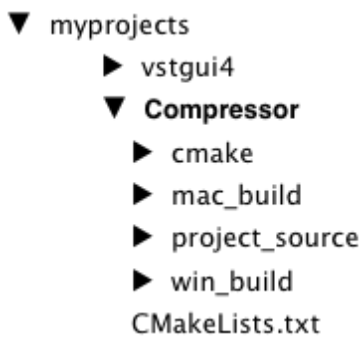
custom GUI or not.

ASPiK Plugin Project Folder

For both the individual and universal API projects, there is only a **single outer folder that contains all of your ASPiK projects along with a single copy of the VSTGUI4 sub-folder**. I use the folder name **myprojects**• for all of my projects, so I will also use it here as an example. Within your outer **myprojects** folder, you need to include the VSTGUI4 sub-folder. A typical **myprojects** folder that contains the VSTGUI4 sub-folder along with three ASPiK projects named Compressor, Reverb, and ParametricEQ would have the following folder hierarchy:



Expanding the Compressor project folder reveals the guts and the all-important CMake setup file:



Setting up your SDKs

SDK Folder Naming Convention

You will need to navigate either Terminal or the Windows Command Prompt to various locations within your SDKs to run CMake. To keep the folder names simple and easy to use with Terminal/Command Prompt, I use a naming convention with:

- no whitespaces (blanks)

- no numbers
- no special characters

There are currently three supported APIs for ASPiK: AAX, AU and VST and each SDK consists of a series of subfolders with AU being the simplest and VST being the most complex in terms of folder hierarchies.

You can setup individual project folders for each API or create a universal folder that will allow you to build for all APIs at once. Here we will set up each SDK independently, and then show the universal SDK folder settings. **Note that you can change this stuff in the CMakeLists.txt file if you know how. Then, you can hardcode whatever folder hierarchy makes sense for you.**

Alternate SDK Folder Structures
If you would like to arrange your SDKs in a different manner you can do that as well but you will need to edit the CMakeLists.txt file to do so and you need to know how to edit CMake files properly. The SDK folder locations are set near the top of the file and are used after the user-editable stuff near the middle. It is strongly advised to first get your ASPiK projects up and running with the folder hierarchy described here. Once you have your first projects compiling properly, you may then want to experiment with alternate SDK folder hierarchies.

Remember the VSTGUI4 Library!
The vstgui4 sub-folder MUST lie within your myprojects folder or you will receive millions of error when you compile mentioning "vstgui" so remeber to add this folder when you are done setting up the SDK outer container folders. See the previous ASPiK Project Folder section.

AU SDK
The AU SDK is rarely updated as it has been stable and effective for many years. In fall 2023, Apple updated the AU SDK to v1.1.0 and began using GitHub to host the SDK. This version of ASPiK requires the **new** v1.1.0 SDK. **There were changes in the new SDK that make the old plugin code incompatible without changing numerous functions**. The AUInstrumentBase was removed so the synth shell code changed to accomodate. However, the changes were fundamentally easy and minimal, and there was no change to the ASPiK core.

NEW USERS
We will setup the AU SDK folder and sub-folders to match the folder hierarchy in the FX and synth plugin books. This way, if you are working from them, all the existing text will be correct except the names of the two sub-folders which are only stated once in the beginning. The original SDK used two sub-folders named **AUPublic** and **PublicUtility**. The new SDK uses two sub-folders named **include** and **src**.

For the AU SDK, you first need to create an outer container for the two SDK sub-folders. You may name this folder whatever you wish - I use the name AU_SDK• for my folder so we will use it here, and this is repeated in the books. And, if you decide to use the Universal API paradigm instead, you can re-use the same AU_SDK outer folder without issues.

Add the myprojects subfolder.

Go to https://github.com/apple/AudioUnitSDK and download the SDK. When you unzip the SDK, it will expose the outer folder named **AudioUnitSDK-main**. Inside of that folder are our target SDK sub-folders, **include** and **src**. Copy these two folders into your AU_SDK folder. Place your **myprojects** folder in parallel with the two SDK sub-folders and inside of your outer AU_SDK folder like this:

- ▼ AU_SDK
  - ▶ include
  - ▶ **myprojects**
  - ▶ src

Then, place the **vstgui4** folder inside of myprojects and you are done.

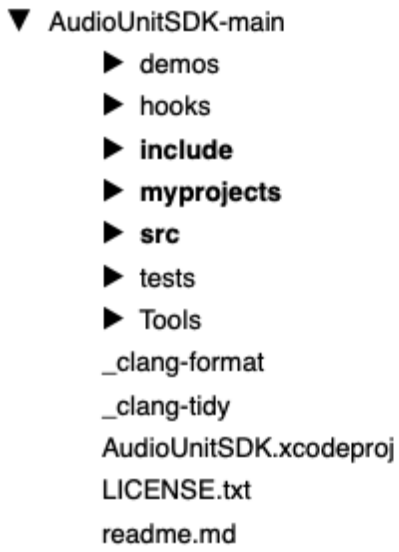
AU v1.0 USERS

If you have already built AU plugins with ASPiK, you have two choices here and you can control this by manually editing a cmake file if you like. The ASPiK default is to merge the old and new SDKs by simply copying the **include** and **src** subfolders into your existing AU_SDK folder. Then, create a new **myprojects** subfolder for the new SDK, and add the new vstgui4 graphics subfolder for this ASPiK SDK version. You will still be able to build both old and new projects from the same folder since the files do not overlap and the new sdkj is namespaced just to make sure. If you use this default MERGED paradigm, your AU_SDK folder will look like this:

- ▼ AU_SDK
  - ▶ AUPublic
  - ▶ **include**
  - ▶ myprojects
  - ▶ **myprojects v1.1.0**
  - ▶ PublicUtility
  - ▶ **src**

STAYING WITHIN THE NEW SDK FOLDER

The second choice for dealing with the new AU SDK is to work within this folder by creating a new **myprojects** sub-folder inside of the main **AudioUnitSDK-main** folder. The new AU SDK has the same twin folder hierarchy as the old one and the relative location of your myprojects sub-folder is all cmake needs so that outer folder name is not important. Your new AU SDK folder will look like this:



If you do not use the Universal build option (build AU, AAX and VST in the same project) then you can do this without modification of anything else and you are done.

If you do intend on using Universal builds, then you will need to tweak one line of script in the outer CMakeLists.txt file for each new project. At the very top just below the # — BEGIN USER INPUT, set the BOOL value to FALSE indicating a NON-MERGED AU SDK folder; NOTE that the default value is TRUE (MERGED).

set(MERGED_AU_SDKS FALSE) # <– set TRUE (default) or FALSE (separate SDKs)

AAX SDK

The AAX SDK is contained within a series of subfolders of the current SDK branch. You must establish an account with Avid to gain access to the SDK and Pro Tools Developer Version and you will need an iLok2 device for storing your Pro Tools activation key.

The AAX SDK architects use numbers when naming the various versions such as AAX_SDK_2p3p0 which reads AAX SDK version 2.3.0• I prefer to rename the current SDK to AAX_SDK in keeping with

folder naming convention discussed in the Intro to CMake portion of the docs, as well as shortening the name a bit. As with the AU projects, your **myprojects** subfolder will be located in parallel with the other SDK sub-folders at the outermost level.And, if you decide to use the Universal API paradigm instead, you can re-use the same AU_SDK outer folder without issues. My folder hierarchy looks like this:

VST SDK

The VST SDK is contained within a series of subfolders of the current SDK branch. The architects name the outermost folder VST_SDK and it contains two inner folders, VST2_SDK and VST3_SDK. In addition, there are two script files, one for Mac and one for Windows. These scripts install the VST2 SDK folders within the VST3 SDK so that you may use the VST2 wrapper that is included with the VST3 SDK.

Important: ASPiK VST projects are VST2 compatible by design and require the VST2 API files to be setup properly. This is fairly simple:

Windows

Navigate to the VST3_SDK folder that contains copy_vst2_to_vst3_sdk.bat, then, double-click on the copy_vst2_to_vst3_sdk.bat file to run the copy mechanism.

MacOS

Open a Terminal window and navigate to the VST3_SDK folder that contains the copy_vst2_to_vst3_sdk.sh file. Drag and drop the copy_vst2_to_vst3_sdk.sh file into Terminal. This will run the copy mechanism.

With the VST2 SDK installed, you can now create the **myprojects** folder for your exported projects. It needs to be located just inside the VST3_SDK subfolder, in parallel with the other SDK files, exactly as with AU and AAX. Your folder hierarchy should look like this:

- VST_SDK
  - VST3_SDK
    - base
    - bin
    - cmake
    - doc
    - **myprojects**
    - pluginterfaces
    - public.sdk
    - vstgui4

You can see the SDK's CMakeLists.txt file - this is entirely independent of your exported projects and when you run CMake for the exported project, it will not interfere in any way with the SDK's files or CMake scripts. You should note that index.html is the Doxygen documentation link.

Universal Exported Project Location

Creating a folder hierarchy for the Universal setup is simple: create an outer folder to hold the SDK sub-folders in parallel with each other. I use the folder name ALL_SDK for my Universal projects. Your myprojects sub-folder is located in parallel with all three SDK folders and is not inside any one of them. My universal project folders look like this:

- ALL_SDK
  - AAX_SDK
  - AU_SDK
  - **myprojects**
  - VST_SDK

Using the ASPiKreator

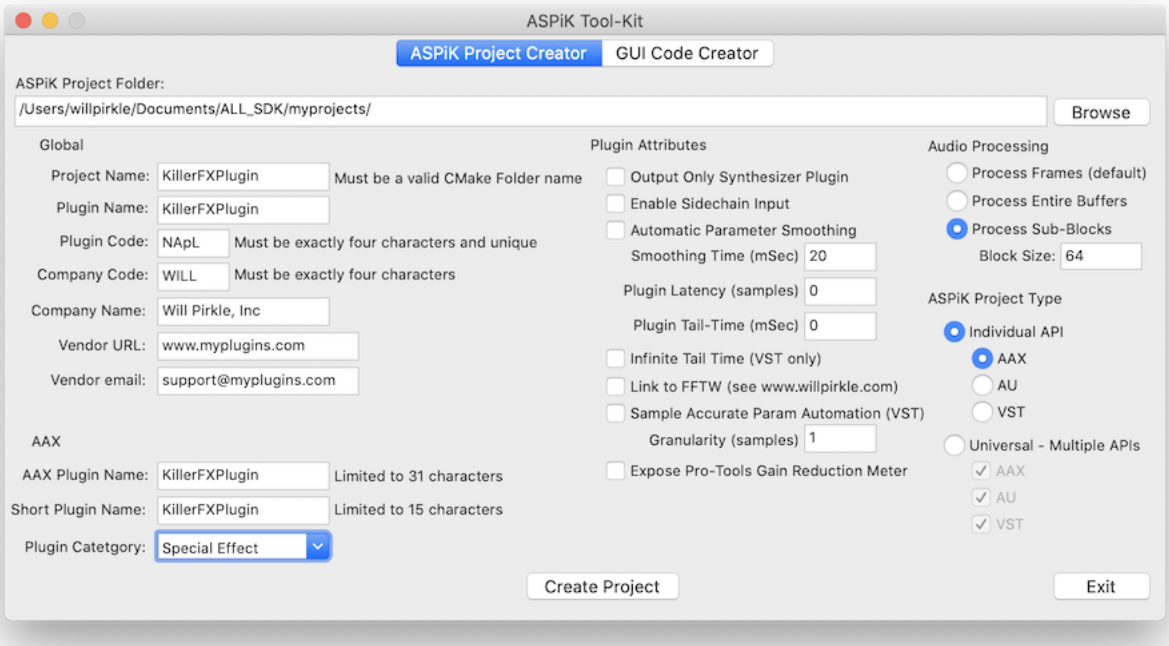
ASPiKreator

Ther SDK comes with a Mac and Windows applicatation named Aspikreator.app and Aspikreator.exe. These apps contain two sub-apps: the **ASPiK Project Creator** and the **GUI Code Creator**.

The ASPiK Project Creator generates new (blank) ASPiK plugin projects for both Mac and Windows.

NOTE: you only need ONE or the other to generate projects for BOTH operating systems; the Mac ASPiK Project Creator can generate the CMake files and projects for Windows and vice-versa. You will need to run CMake on the target operating system after the projects are created. We will discuss the GUI Control Creator in the section that involves writing your first plugin.

The main panels for both Mac and Windows versions have identical controls and the Mac version is shown here for a plugin project named ASPiKompressor:



Using the ASPiKreator software is super-easy. You only need to set a few text fields and check a few boxes to generate new plugin projects.

Project/Plugin Naming

The project name will become the name of the plugin file; e.g. if the project is named ASPiKompressor, then the AAX plugin will be named ASPiKompressor.aaxplugin, the VST will be ASPiKompressor.vst3, and so on. However, the plugin APIs also specify a plugin name string that is independent of the acutal file name. BUT, different DAWs will expose the plugin names to the user in different manners - some will use the filename while others will use the internal name string. The simplest solution is to name the plugin and the project identically, however the choice is yours and you can always tweak this in the individual plugin shell code, to which you have full access!

Step 1: Setup the Target Folder

Choose the destination ASPiK project folder as described on the ASPiK Project Folder page. For Universal projects, you will almost always be writing to the same project folder. For individual projects, you will likely need to change this folder. Use the [Browse] button to locate the folder. In this case, we are creating a universal plugin project so we use the .../ALL_SDK/myprojects folder as seen in the figure above.

Step 2: Set your Project Attributes

Fill in the text fields for the various components and choose the AAX category; check all the boxes that apply to your project. Select the type of project (universal or individual) and choose the APIs to include. Make sure you set the **Plugin Code to a 4-character sequence unique among YOUR products**.

Step 3: Choose the Audio Processing

Starting with ASPiK 1.6.8, you may now setup the project to process audio in three different ways (this has always been available, but required the user to add this code - now that is done for you). .

1. **Process Frames**: this is the default mechanism that sets up processing on each sample interval so that you process one sample per channel at a time; this is conceptually the simplest (great for newbies) but is the least efficient on the CPU.
2. **Process Entire Buffers**: in this paradigm, you process entire buffers directly from the DAW's buffer pointers; each channel has its own buffer (explained in my FX plugin book in high detail). This is the most CPU efficient and will produce blazingly fast code.
3. **Process Sub-Blocks**: here, the incoming DAW buffers are broken into sub-blocks with a block-size that you prescribe. You still operate directly on the DAW's buffers (so it is also very efficient) but you operate on chunks of audio at a time; this is a common paradigm for synth plugins, or if you are working with FFTs of a set length. NOTE: if a partial block arrives from the DAW (this used to be common, but is now somewhat rare) then the entire partial buffer is sent to your processing function.

Note: The **PluginCore** already contains two example sub-block processing functions named **renderFXPassThrough** and **renderSynthSilence** that demonstrate how to iterate over the sub-blocks for processing as well as setting up the GUI parameter transfers (cooking functions) for your objects.

Step 4: Create the Project

Use the [Create Project] button to spawn off a new project folder and setup the CMakeLists.txt file with the values from the ASPiKreator interface. With the new project in place, you only need to run CMake to generate the Visual Studio and/or XCode compiler project to build your plugins.

Step 5: Save your Settings

Use the [Exit] button to quit ASPiKreator - when you use this button (and not the X close toolbar

button) your settings will be saved for: company name, 4-char company ID, company URL, company email, and your ASPiK Project folder. Note that all of the other attributes must be set for each plugin project (many items are necessarily different).

Step 5: OPTIONAL - Customizing Project Creation: Template Source
The ASPiKreator uses a special copy of the PluginTemplate folder that you find in the SDK. This special copy is slightly different in that the CMakeLists.txt file includes special text placeholders, and the ASPiKreator softare uses them to fill in the file with your settings. However the rest of the PluginTemplate is unmodified. If you want to modify some of the code in this template so that each new project includes your additions, then you may do that by simply modifying the files that the ASPiKreator uses. You may even add files to the template so that they will be automatically copied to each new project, but you will need to manually add them to the compiler if they are source files.

MacOS

The Template Source is located within the ASPiKreator bundle itself; right click on the application and choose "Show Package Contents" then browse to the Resources folder, where you will find the special template folder that you may modify

Windows

The Template Source is located within the ASPiKreator folder that you find in the C:\Program Files folder, in parallel with the ASPiKreator.exe binary. You may modify this template folder as you wish, but you will need admin privaleges to do so.

Optional: Manual Project Creation

DIY Project Creation
If you'd like to create your plugin project by hand, you can do that with the SDK as well. The steps are outlined in Designing Audio Effects Plugins in C++ 2nd Ed. by Will Pirkle and are repeated here.

Step 1: Copy the TemplatePlugin
Copy and paste the TemplatePlugin folder into your ASPiK project folder. Change the name of the folder to a new name that makes sense for the project. Make sure it is a valid CMake name (no spaces, no oddball characters, no numbers).

Step 2: Edit the CMake File

Open the outermost CMakeLists.txt file in any text editor and alter the variables at the top of the file according to your plugin needs. In the example here, we are setting up a project for the book's IIRFilters plugin:

At the top of the file are the names of the SDK root folders and they will be preset to the names we used in Section 6.2 and will look like this:

```
# --- Here you can set the names of your UNIVERSAL SDK folders <br>
set(UNI_AAX_SDK_FOLDER AAX_SDK)
set(UNI_AU_SDK_FOLDER AU_SDK)
set(UNI_VST3_SDK_FOLDER VST_SDK/VST3_SDK)
```

These folder names are only needed for universal projects where you target one or more SDKs and you use the outside myprojects folder in Section 6.2.5. If you are targeting individual APIs, you don't need to bother with the naming as CMake uses relative path locations. The next section contains the build flags. Here is where you choose which APIs to target for the compiler. The settings below show a VST-only build setting. The UNIVERSAL_SDK_BUILD flag is set to FALSE (for an individual build) and the VST_SDK_BUILD flag is set to TRUE, indicating that API as the target.

```
# --- Universal Build Flag - when using combined SDKs
set(UNIVERSAL_SDK_BUILD FALSE) # <-- set TRUE or FALSE
# Individual project builds<br>
set(AAX_SDK_BUILD FALSE) # <-- set TRUE or FALSE
set(AU_SDK_BUILD FALSE) # <-- set TRUE or FALSE
set(VST_SDK_BUILD TRUE) # <-- set TRUE or FALSE
```

To build a Universal project that targets VST and AU, you would set these flags accordingly:

```
# --- Universal Build Flag - when using combined SDKs
set(UNIVERSAL_SDK_BUILD TRUE)# <-- set TRUE or FALSE
# --- Individual project builds
set(AAX_SDK_BUILD FALSE) # <-- set TRUE or FALSE<br>
set(AU_SDK_BUILD TRUE) # <-- set TRUE or FALSE<br>
set(VST_SDK_BUILD TRUE) # <-- set TRUE or FALSE
#
```

With the build flags set, you can finish the edits with the project information. These values are set in the next section of the file and are generally self-explanatory. The first is the compiler project name (IIRFilters) and this project name must not contain whitespaces or special characters.

```
# --- the name of the project must be valid CMAKE name: set PROJECT NAME with or
      without quotes
set(PLUGIN_PROJECT_NAME IIRFilters)
#
```

The next edit is for the plugin name that the DAW will expose to the user; here we are calling it the Filterizer. Notice that this name contains quotes and can be the same as or different than the iroject name. You can include whitespaces, numbers, other characters, or almost anything else in this text description.

```
# --- the name of the PLUGIN as you want it to appear in DAW: set PLUGIN NAME with
      quotes<br>
set(PLUGIN_NAME "Filterizer")
#
```

The next edit is for identifying the plugin as a FX or Synth plugin. It uses a boolean flag to identify the synth-ness of the project. A synth plugin only has outputs and does not process an audio input (unless it is a sidechain). All projects in this book are FX plugins.

```
# --- Synth Flag: FALSE = FX plugin, TRUE = Synth Plugin<br>
set(IS_SYNTH_PLUGIN FALSE) # <-- set Synth Flag
#
```

Then, you need to fill in your vendor (company) information

```
# --- VENDOR information (your company stuff)
set(VENDOR_NAME "Plugin Makers") # <-- required!<br>
set(VENDOR_URL "www.yourcompany.com") # <-- OPTIONAL (ok if blank)<br>
set(VENDOR_EMAIL "help@yourcompany.com")# <-- OPTIONAL (ok if blank)
```

Next, you need to set some special four-character codes that are required for identification in the various APIs. You need to be careful here, and be sure to change the codes for each plugin you develop. You can use combinations of capital and lower case letters and numbers or other characters.

```
# --- 4-Character Codes (for AAX and VST3): MUST be unique for each plugin your
      company creates/sells<br>
set(4CHAR_CODE "IIR1")
# --- your company CODE (does NOT need to be unique for each plugin)<br>
set(4CHAR_MANUFACTURER_ID "PLMK")
# --- MUST be unique for each AAX plugin your company creates/sells<br>
set(4CHAR_AAX_PRODUCT_ID "iir1")
#
```

Lastly, there are some plugin options that need to be set. First is the audio processing (see **Frame, Buffer and Sub-Block Processing**); here I am settign up sub-block procssing of 64 samples at a time:

```
# --- Plugin Options ---
set(PROCESS_FRAMES FALSE) # <-- set TRUE or FALSE
set(PROCESS_BUFFERS FALSE) # <-- set TRUE or FALSE
set(PROCESS_BLOCKS TRUE) # <-- set TRUE or FALSE
set(BLOCK_SIZE 64) # <-- numerical, in samples (per channel) if using blocks
```

```
###
```

The next option is for side chaining. If you would like the DAW to expose side chain inputs to your plugin, set the EXPOSE_SIDECHAIN flag to TRUE, otherwise false. Our volume plugin won't need it so we will set it to FALSE here.

```
# --- expose SC<br>
(EXPOSE_SIDECHAIN FALSE) </strong># <-- set TRUE or FALSE
#
```

Next is the latency setting in samples. This is a special setting for lookahead compressors or FFT processing plugins that necessarily introduce a latency or time-delay to the audio signal. Set this parameter in samples.

```
# --- LATENCY for lookahead plugins<br>
set(LATENCY_IN_SAMPLES 0) # <-- numerical, in samples
#
```

The tail time may be set so that you can hear your reverb or delay tails. For VST3, there is an option for an infinite tail – it this variable is set to true, the DAW will run a continuous stream of zeros through your plugin forever after the user stops playback the very first time. We'll use a default tail time of 1 second here, even though our volume plugin won't need a tail and we'll set the VST3 infinite tail to FALSE.

```
# --- tail-time for reverb/delay plugins<br>
set(TAIL_TIME_MSEC 1000) # <-- numerical, in mSec
# --- VST3 options: infinite tail (DAW keeps streaming forever)<br>
set(VST3_INFINITE_TAIL FALSE)
#
```

The last option involves sample accurate automation for VST3 plugins only. For our IIRFilters plugin, we will leave this in their default settings of no sample accurate automation and a granularity of 1 sample.

```
#
set(VST3_SAMPLE_ACCURATE_AUTOMATION FALSE)
set(VST3_SAMPLE_ACCURATE_GRANULARITY 1)
#
```

There are three more flags to set – one will automatically include the **fxobjects.h** and **fxobjects.cpp** files to integrate the book C++ objects into your project. The next is for linking with the FFTW library for the advanced plugins in Chapters 20 and 22. Note that you must install FFTW on your system prior to setting the link flag.

```
# --- more flags <br>
set(INCLUDE_FX_OBJECTS TRUE) # <-- set TRUE or FALSE<br>
```

```
set(LINK_FFTW FALSE) # <-- set TRUE or FALSE<br>
#
```

AAX Plugin Category

The AAX plugin category is coded as an unsigned integer and based on a list of plugin codes. If you use the ASPiKreator software, you can see this list in the drop-down combo box. The encoding from the documentation is:

AAX_ePlugInCategory_None = 0x00000000,
AAX_ePlugInCategory_EQ = 0x00000001, ///< Equalization
AAX_ePlugInCategory_Dynamics = 0x00000002, ///< Compressor, expander, limiter, etc.
AAX_ePlugInCategory_PitchShift = 0x00000004, ///< Pitch processing
AAX_ePlugInCategory_Reverb = 0x00000008, ///< Reverberation and room/space simulation
AAX_ePlugInCategory_Delay = 0x00000010, ///< Delay and echo
AAX_ePlugInCategory_Modulation = 0x00000020, ///< Phasing, flanging, chorus, etc.
AAX_ePlugInCategory_Harmonic = 0x00000040, ///< Distortion, saturation, and harmonic enhancement
AAX_ePlugInCategory_NoiseReduction = 0x00000080, ///< Noise reduction
AAX_ePlugInCategory_Dither = 0x00000100, ///< Dither, noise shaping, etc.
AAX_ePlugInCategory_SoundField = 0x00000200, ///< Pan, auto-pan, upmix and downmix, and surround handling
AAX_ePlugInCategory_HWGenerators = 0x00000400, ///< Fixed hardware audio sources such as SampleCell
AAX_ePlugInCategory_SWGenerators = 0x00000800, ///< Virtual instruments and other software audio sources
AAX_ePlugInCategory_WrappedPlugin = 0x00001000, ///< All plug-ins wrapped by a third party wrapper
AAX_EPlugInCategory_Effect = 0x00002000, ///< Special effects

```
#
set(AAX_CATEGORY 0) # <-- see AAX Category codes in ASPiK SDK Documentation
#
```

That’s all there is to editing this file unless you want to make changes regarding the relative locations of the SDK folders or other tweaks. Be sure to visit www.willpirkle.com for these instructions if you are interested. Now you can save the file and we can run CMake to finish off the project process.

-

Running CMake

CMake
This document explains how to install and run CMake on your ASPiK projects. It also includes specifics to each API including the location of your final plugin once it is built.

Installing CMake

In order to generate the ASPiK project compiler files, you need to have CMake installed on your OS. For ease, you should add the CMake executable to your default path. Download the latest version of CMake at http://cmake.org/

Windows
During installation, you have the option of adding CMake to your default path list.
Choose YES for this option and you are done.

MacOS
Open Terminal and type:

echo $PATH

which produces a colon : separated list of folders you can run directly with terminal;

/usr/local/bin

should be in this list

Next, add a SymLink to the CMake executable and place it in the /usr/local/bin folder in Terminal by running:

sudo ln -s /Applications/cmake.app/Contents/bin/cmake /usr/local/bin

After that, you are done.

In each case (Windows vs MacOS), you first navigate to the <OS>_build folder inside of the project folder. For Windows it is named **win_build** and for MacOS it is named **mac_build**. You will notice that these folders are empty when you navigate to them. This is correct for a freshly created project.

Running CMake will populate the build folder with the newly created XCode or Visual Studio project. After that you can open the compiler project and rebuild your solution as usual. To run CMake:

MaxOS: open a Terminal shell

WinOS: open the Command Prompt with admin privileges (aka "Command Prompt (Admin)")
Navigate and find your myprojects folder inside a particular SDK (individual) or in the ALL_SDK folder (universal).

Navigate into your ASPiK project folder and then again into either the win_build or mac_build sub-folders according to your OS. You must run CMake from folder *inside* of the one that contains the outer CMakeLists.txt file. This is a quirk of CMake.

Run CMake as follows:

VS 2015 (for VST3 and AAX) 64-bit:
cmake -G"Visual Studio 14 2015 Win64" ../

VS 2017 (for VST3 and AAX) 64-bit:
cmake -G"Visual Studio 15 2017 Win64" ../

VS 2019 (for VST3 and AAX) 64-bit:
cmake -G"Visual Studio 16 2019" ../

Xcode:
cmake -GXcode ../

You can find the CMake generator (G) codes for other Visual Studio compilers here:

https://cmake.org/cmake/help/v3.10/manual/cmake-generators.7.html

NOTE: the VSTGUI SDK requires C++11 so make sure your compiler is 100% C++11 compliant. Many older versions of Visual Studio are NOT 100% compliant so beware.

Compile Targets
Xcode calls each project a "target" • while Visual Studio uses "Project" each of these compiles to produce either your final plugin or a library (or multiple libraries) needed to compile it.

For individual projects, you will only have target(s) associated with one particular API. For universal projects, there will be targets for everything required to build for all APIs.

VST3: The VST3 compiler project will include targets for two executables: validator and editorhost.

These are automatically added by a CMakeLists.txt file that is internal to the VST3 SDK and that we do not want to modify. The validator project is required in the solution. The editorhost is optional and you can safely ignore it. See the VST3 section below for more information about the validator.

-

AAX Dev Notes

Supported OS & Compiler

Windows:

OS: Windows 8.1 and Windows 10

Compilers: Visual Studio 2015 (AVID approved) or Visual Studio 2017 (Not fully AVID tested but works)

MacOS:

OS: 10.11 and higher

Compilers: Xcode 7 and higher

AAX Parameter Count

The AAX monolithic parameters paradigm requires a queue of parameters for tight sync with the GUI and audio. This is part of a static defintion and can't be changed programmatically after the object has been created. The definition of the maximum number of parameters is in the **AAXPluginParameters.h** file and is set to a value of 64. If your plugin needs more than 64 parameters, you need to change this value (OK if this is a large number, but it needs to be changed before compiling). See:

#define kSynchronizedParameterQueueSize 64

in the **AAXPluginParameters.h** file if you need to change it!

AAX Library

You must compile the AAX Base Library first. You only need to do this once per SDK release (or if you update your compiler to a newer version). You can find the XCode and Visual Studio generated library projects in these folders:

AAX_SDK/Libs/AAXLibrary/WinBuild

or

AAX_SDK/Libs/AAXLibrary/MacBuild

Open the project file and compile your project using the same compiler that you will use to compile your project. Anytime you change compilers, you must re-compile the library with your new compiler. The result of the library compilation will be:

MacOS
Starting with SDK 2.3.1, there are two separate MacOS/Xcode library targets depending on the use the std:: library. ASPiK (and all of the AAX demo plugins) require the "_libcpp" versions of the libraries so choose the correct target when building these libs.
AAX_SDK/Libs/Debug/libAAXLibrary_libcpp.a
AAX_SDK/Libs/Release/libAAXLibrary_libcpp.a

Windows x64
AAX_SDK/Libs/Debug/AAXLibrary_x64_D.lib
AAX_SDK/Libs/Release/AAXLibrary_x64.lib

Open the compiler project for your newly generated project. With these libraries built, you can now re-build the solution to complete the ASPiK project.

Finding your Plugin
Your finished plugin will be located in the following sub-folder:

AAX_SDK/myprojects/(project name)/mac_build/AAX/(config)

or

AAX_SDK/myprojects/(project name)/win_build/AAX/(config)

where (config) is either Debug or Release, depending on how you built the project.

Copy the .aaxplugin into your AAX plugin folder. You must to this manually because in both operating systems, the target folder is admin-protected which prohibits the copy mechanism from being done programmatically during compilation.

MacOS
/Library/Application Support/Avid/Audio/Plug-Ins

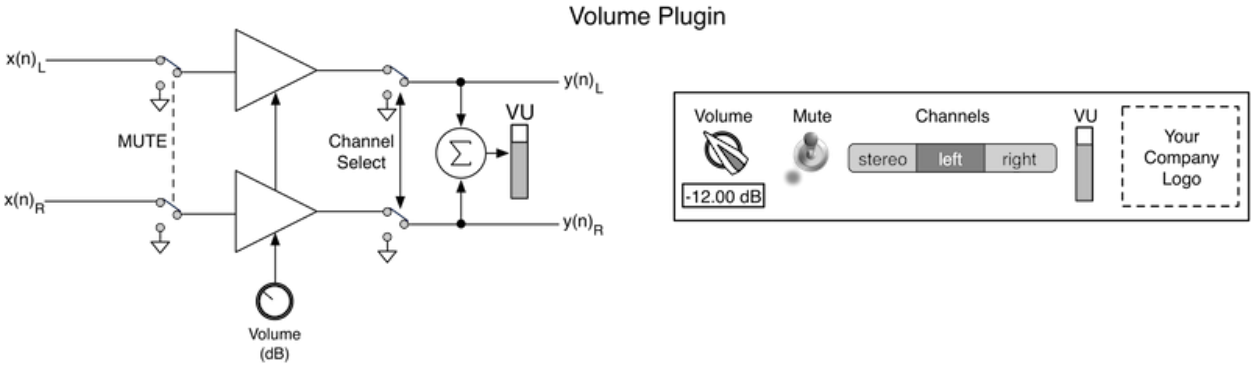
Windows

C:\Program Files\Common Files\Avid\Audio\Plug-Ins

Now you can run your Pro Tools Developer's Build to test your plugin and generate the Presets for it.

-

- # Plugin Specifications
- **Block Diagrams**
  The plugin specification usually starts with block diagrams. Let's look at the block diagram and the accompanying GUI that we will implement. This glorified volume plugin isn't going to win any technical awards, but it will show you how to write all the code you need and design the GUI as well.



-
-

  The block diagram on the left will be used for coding the audio processing - we identify the various blocks and connections an implement these in code. If coding audio block diagrams is a new concept, the you are encouraged to check out **Designing Audio Effects Plugins in C++**D 2nd Ed. by Will Pirkle as it explains how the block diagrams turn into code. Therefore, we won't go into detail on that aspect of the process.

  The GUI mockup on the right will help us assemble the final GUI for the user. It will also help solidify the notion that the GUI controls connect to the block diagram. Specifically, the GUI control information needs to propagate into the plugin (the ASPiK framework handles this for you), and we will need to store and use that information in a DSP algorithm. **The ordinary way to accomplish this is to map the GUI controls to C++ variables in the plugin core object**. ASPiK features Automatic Variable Binding to allow this transfer of information from the GUI into the plugin variables to occur transparently to you, the programmer. Note that this is an optional feature, and you are free to implement this binding process on your own if you like.
- # GUI Control Tables

- Making a GUI control table will help you list the GUI controls and the kinds of parameters they represent (see the ASPiK Framework API for all the details). For our plugin, we need to define some input parameters that will usually represent data arriving from the GUI. We have already decided on some variable to map these controls into and listed those in the table as well. Note also that plugin parameters do not necessarily need to come from the GUI - for example the parameters may be hardcoded, or may be defined in an external file that the plugin reads.

Control Name	Units	min/max/default or string-list	Taper	Linked Variable	Linked Variable type
Volume	dB	-60.0 / +12.0 / 0.0	linear	*volume_dB*	double
Mute	-	{OFF, ON}		*enableMute*	int
Channel Select	-	{stereo, left, right}		*channels*	int

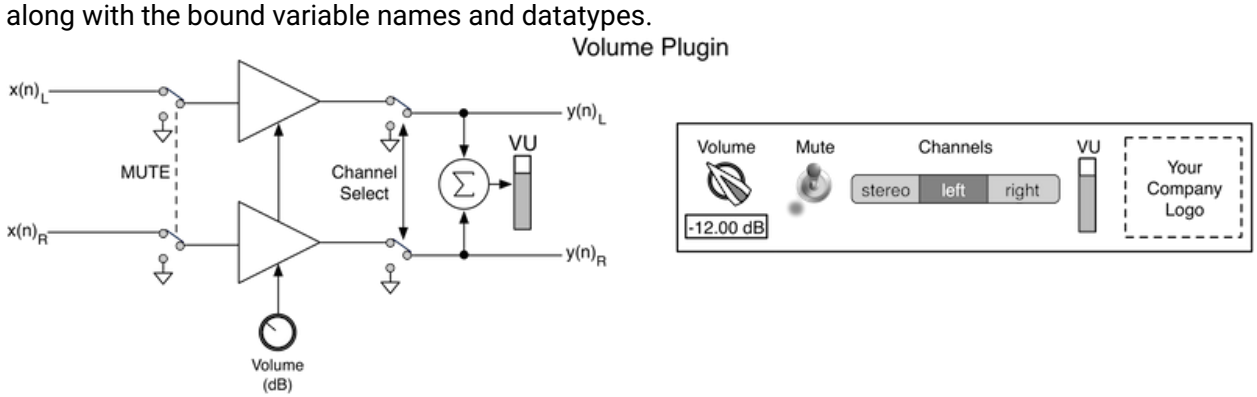
-
-

We also need to define one output parameter representing the VU meter on the GUI. We will need to form the value for the meter and send it back out to the GUI at the end of the buffer processing cycle. Note that ASPiK meter parameters allow you to specify the meter ballistics, that is, the attack and release time of the meter's motion. All ASPiK meter parameter objects include a built-in audio detector object that allows peak, MS, and RMS detection choices along with linear or logarithmic display output. Notice how these attributes manifest themselves in the table.

Meter Name	Inverted	attack / release	Detect Mode	Calibration	Variable Name
VU	no	10.0 / 500.0	RMS	logarithmic	*vuMeter*

-
-

Make sure you can match up the GUI controls and their associated entries in the table along with the bound variable names and datatypes.



-
-

Enumerating the Plugin Parameters

Each plugin parameter (GUI control) must be identified with some kind of unique value - this is common across all plugin APIs. ASPiK uses unique **integer** values (so do AU and VST3 while AAX uses unique character strings). It is up to you as to how you want to define your unique integer variables. We have come up with an easy way to make these definitions without having to memorize more stuff. Since each GUI control will be bound to a C++ member variable on a plugin object (for ASPiK this is the **PluginCore** object), then those member variable names must be unique. We can define an enumeration that uses the same variable names and add an enumeration prefix to make it easy to specify the unique values.

This is one of the things that you will need to code by hand, but it is quite simple. The reason that the ASPiK Tool-Kit doesn't attempt to code this information for you is that most programmers have their own methods for making unique enumerations and may even have preset enumeration files from other projects.

plugincore.h

For ASPiK, the **PluginCore** object is the monolithic plugin device. It is coded in the **plugincore.h** and **plugincore.cpp** files. You will spend the majority of your time modifying these two files. In fact you can write the entire plugin within just these two files if you like. Open the **plugincore.h** file and add an enumeration at the top of the file:

```
//
enum controlID {
volume_dB,
enableMute,
channels,
vuMeter };
//
```

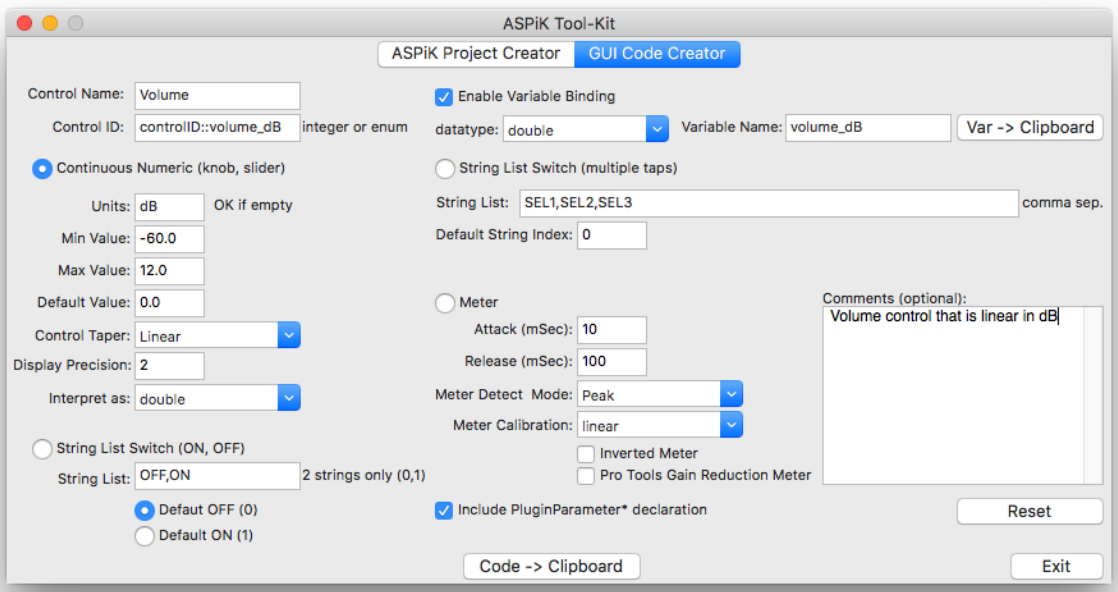
Notice that we have defined an enumeration prefix *controlID* along with the enumerated variables that exactly match the variable names in the GUI Control Tables. This means that you can access the unique identifier for your *enableMute* variable as *controlID::enableMute* rather than the underlying integer (1) in this case.

With the enumeration set up, you can proceed to define the **PluginCore** member variables that map to the GUI controls. **You can do this at the same time as you define your plugin parameter objects using the ASPiK Tool-Kit's GUI Code Creator.**

-

Coding the Plugin Parameters with Variable Binding

At this point, you can open up the ASPiK Tool-Kit application that is part of the SDK and use the **GUI Code Creator** to generate the C++ code for both the bound variable as well as the PluginParameter declaration and instantiation. **NOTE:** this is optional and you may always code the plugin parameters by hand. The ASPiK **GUI Code Creator** is useful for starting out and you can always tweak and modify the code it gerenates. Open the ASPiK Tool-Kit and switch to the GUI Code Creator. The first entry in the GUI control table is for the continuous Volume in dB control (this will map to a continous knob control on the GUI). Select the "Continuous Numeric" control type and fill in the information for the PluginParameter and the bound variable from the table. Notice how the variable name "volume_dB" matches the unique control identifier which also matches the previously declared enumeration. Also, since this is our first parameter, we have c checked the "Include PluginParamater* declaration" box - you only need to do this once for the first parameter you define.



With this data set, you can now generate the C++ code and paste it into your plugin project:

Bound Variable Code:
goes in the plugincore.h file in the "private" area provided for your member variables; click the [Var -> Clipboard] button to copy the code to the clipboard, then paste it into the plugincore.h file; note that it will add a default declaration value based on the default value you declared in the GUI Code Creator app.

```
//
class PluginCore : public PluginBase
```

```
{
public:
PluginCore();
virtual ~PluginCore(){}
// SNIP SNIP SNIP...
private:
// --- Continuous Plugin Variables
double volume_dB = 0.000000; // <-- bound member variable!
// etc...
```

PluginParameter instantiation Code:

goes in the **PluginCore::initPluginParameters( )** function at the top just after the if (pluginParameterMap.size() > 0) statement. Click the [Code -> Clinpbaord] button to copy the code to the clipboard, then paste it into the **PluginCore::initPluginParameters( )** function;

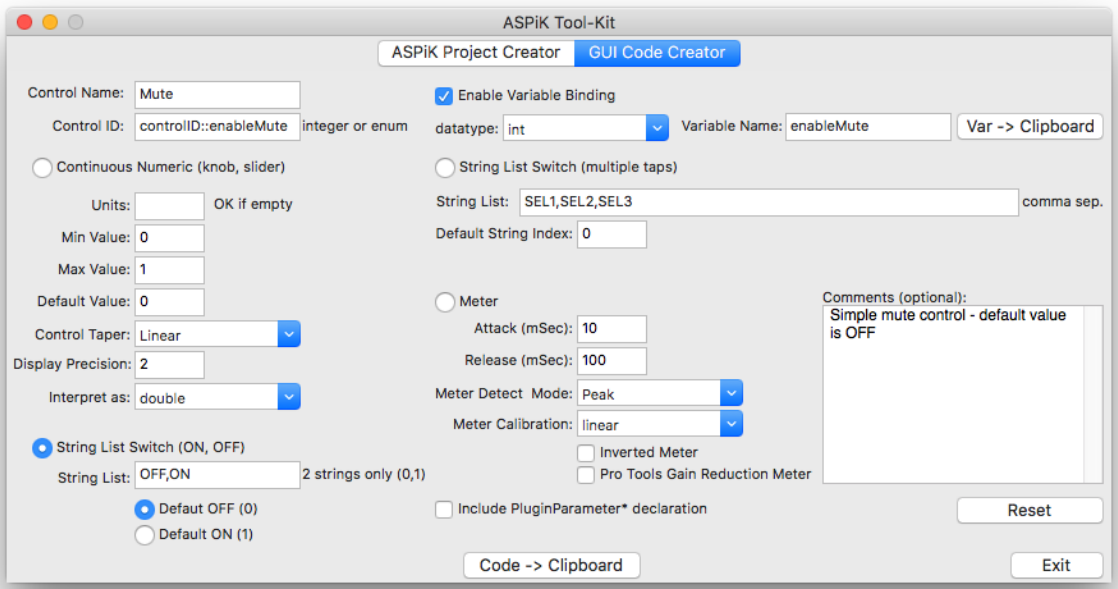
```
//
bool PluginCore::initPluginParameters()
{
if (pluginParameterMap.size() > 0)
return true;
PluginParameter* piParam = nullptr;
/* Volume control that is linear in dB */
piParam = new PluginParameter(controlID::volume_dB, "Volume", "dB",
controlVariableType::kDouble, -60.000000, 12.000000, 0.000000,
taper::kLinearTaper);
piParam->setBoundVariable(&volume_dB, boundVariableType::kDouble);
addPluginParameter(piParam);
etc...
//
```

Parameter Smoothing

ASPiK features automatic parameter smoothing for continuous controls. It can be applied to any continuous control and is a local option, that is, it can be applied to individual parameters (or not at all) and parameters can have different parameter smoothing times (see the Pirkle text for much more information). Most programmers will use the same smoothing time for the majority of their controls. Therefore, parameter smoothing is set on the ASPiK Project Creator panel as a sort of global option. To use it, just open the ASPiK Project Creator panel and check the box along with a default smoothing time in milliseconds. You don't need to create a new project or anything - the parameter smoothing time is the only data that carries over into the GUI Code Creator panel. If you set up parameter smoothing you will have a few more lines of code added - note that you may manipulate the functions later to turn off parameter smoothing or change the smoothing time. These additional lines will be added after the instantiation:

```
//
piParam->setParameterSmoothing(true);
piParam->setSmoothingTimeMsec(20.000000);
//
```

Now you can add the other member variables. Next up is the Mute switch; it is implemented as a String-List parameter with two strings: OFF and ON that determine its state. Hit the [Reset] button on the **GUI Code Creator** to clear out the last entry. Then, select the "String List Switch (ON, OFF)" parameter type. In this case, we will use the default string list of OFF,ON and set the default to the OFF (0) state. Note that you can name the pair of strings however you like (e.g. ENABLED, DISABLED or OUI, NON) knowing that the first string will map to the value (0) while the second maps to (1). Notice also that we've de-selected the "Include PluginParameter*..." switch.



You can now add the bound variable to the plugincore.h file and the parameter instantiation code to the plugincore.cpp file as before, pasting the code in sequence. First, notice how the String-List bound variable declaration resulted in two lines of code; one for the GUI control receiver variable (enableMute) and the other as a strongly typed enumeration that uses the variable name concatenated with "Enum" - this is to allow you to easily evaluate the state of the GUI control item using the built in MACROs that are listed in the SDK. You may decide to use other methods to identify the state of the receiver variable if you wish, but we use the strongly typed enumeration method in the sample code. In ASPiK, String-List controls are ALWAYS bound to variables with the **int** data-type and using the [Var -> Clipboard] feature will always bind an integer type regardless of what you choose in the drop-list.

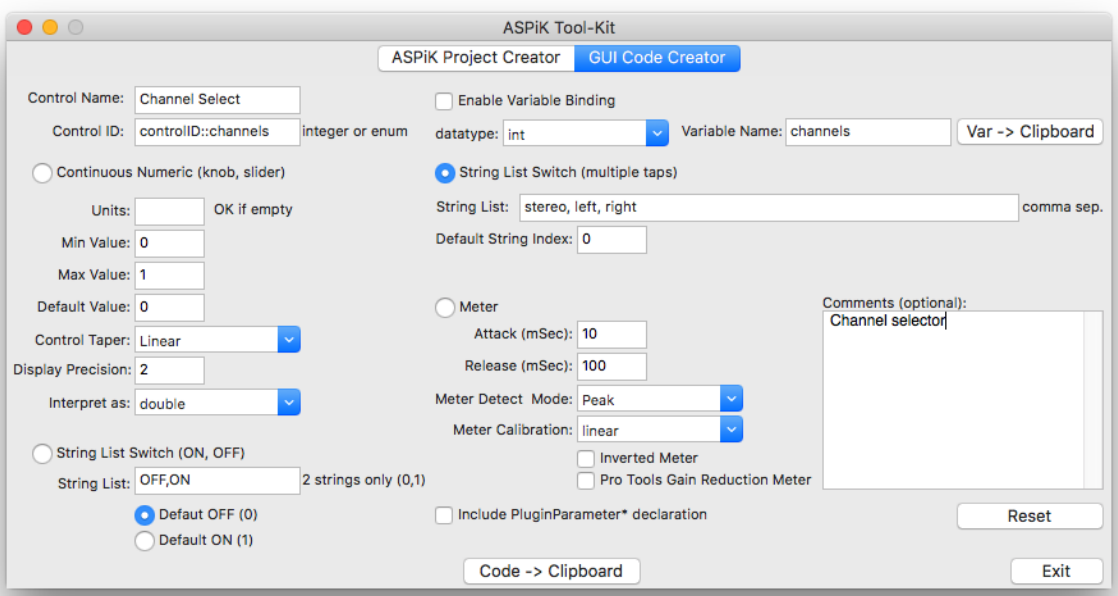
```
//
private:
// --- Continuous Plugin Variables
```

```
double volume_dB = 0.000000;
// --- String-List Plugin Variables
int enableMute = 0;
enum class enableMuteEnum { "OFF,ON" };
//
```

Next, you can paste the code for the string list parameter just below the instantiation for the first parameter:

```
//
/* Simple mute control - default value is OFF */
piParam = new PluginParameter(controlID::enableMute, "Mute", "OFF,ON", "OFF");
piParam->setBoundVariable(&enableMute, boundVariableType::kInt);
addPluginParameter(piParam);
//
```

Next up is the channel selection GUI control. This is done with another String-List, but this one will need more than just two states. Reset the GUI Control Creator and select the "String List Switch (multiple taps)" parameter, then enter the information from the GUI control table. Notice that you specify the default value with an unsigned integer index - the list is zero-indexed, so the first item is number 0, the second is 1, etc...



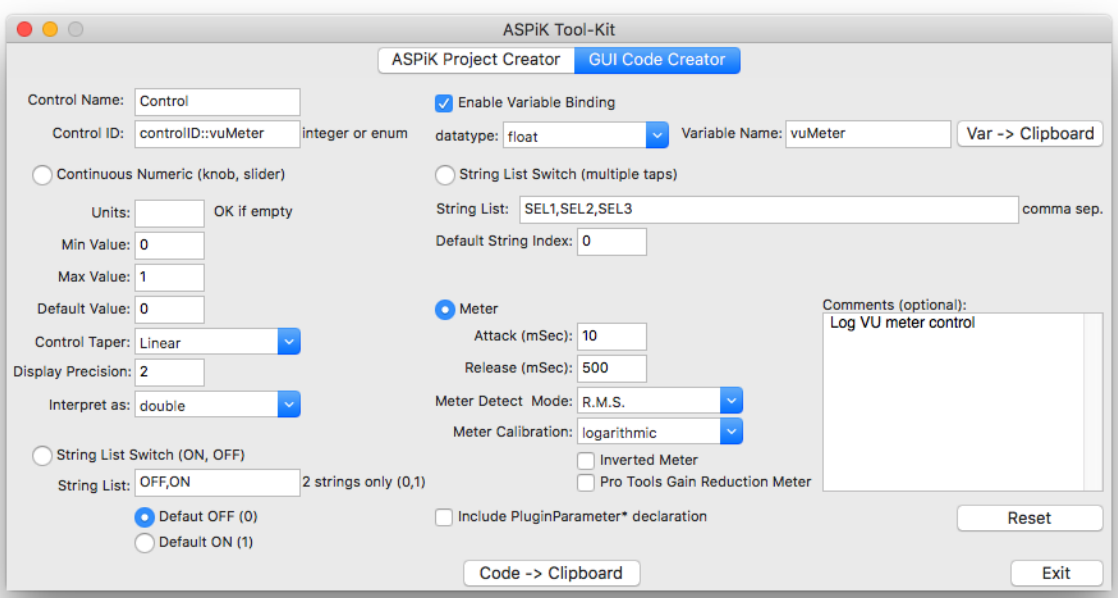
Once again use the [Var -> Clipboard] and [Code -> Clipboard] buttons to generate and paste the code into your **plugincore.h** and **plugincore.cpp** files:

```
//
private:
// --- Continuous Plugin Variables
double volume_dB = 0.000000;
// --- String-List Plugin Variables
int enableMute = 0;
enum class enableMuteEnum { "OFF,ON" };
int channels = 0;
enum class channelsEnum { "stereo, left, right" };
//
```

Then, you can paste the code for the string list parameter just below the instantiation for the first parameter:

```
//
/* Channel selector */
piParam = new PluginParameter(controlID::channels, "Channel Select", "stereo, left,
      right", "stereo");
piParam->setBoundVariable(&channels, boundVariableType::kInt);
addPluginParameter(piParam);
//
```

Our last parameter to encode is an output parameter in the form of a VU meter on the GUI. In ASPiK, meter controls are ALWAYS bound to variables with the ***float*** data-type and using the [Var -> Clipboard] feature will always bind a floating point type regardless of what you choose in the drop-list. Once again reset the GUI Control Creator panel and select the "Meter" parameter type, then add the information from the GUI control table like before.



Once again use the [Var -> Clipboard] and [Code -> Clipboard] buttons to generate and paste the code into your **plugincore.h** and **plugincore.cpp** files:

```
//
private:
// --- Continuous Plugin Variables
double volume_dB = 0.000000;
// --- String-List Plugin Variables
int enableMute = 0;
enum class enableMuteEnum { "OFF,ON" };
int channels = 0;
enum class channelsEnum { "stereo, left, right" };
float vuMeter = 0.0;
```

Then, you can paste the code for the VU meter parameter just below the instantiation for the last parameter:

```
//
/* Log VU meter control */
piParam = new PluginParameter(controlID::vuMeter, "Control", 10.00, 500.00,
       ENVELOPE_DETECT_MODE_RMS, meterCal::kLogMeter);
piParam->setInvertedMeter(false);
piParam->setIsProtoolsGRMeter(false);
piParam->setBoundVariable(&vuMeter, boundVariableType::kFloat);
addPluginParameter(piParam);
//
```

At this point, all of the plugin bound variables and plugin parameters have been declared and defined. We can now move on to the plugin's reset operation.

Plugin Reset

Before getting into details about audio processing, we need to think about what happens after the user loads the plugin, but before audio data streams through it. One of the most important pieces of information we need is the current sample rate of the audio that will be processed through the plugin. Knowing the sample rate is critical for many of our calculations. This usually includes filter coefficients and delay-buffer sizes. The various API's each provide a mechanism for retrieving the current sample rate and this will depend on the API manufacturer. In some systems, the sample rate might change during the course of a DAW session and in others it must be locked down and set prior to the sessions creation. In all cases, the sample rate will not change while the audio is streaming into the plugin.

The **PluginCore** implements a reset function that is called during the plugin initialization, and then again any time the sample rate changes. It is also called in some APIs if the user stops and then restarts audio playback. There is some grey area here as well, based on how the DAW designers interpret the plugin API, or how the DAW itself operates and handles files with different sample rates. However, we can be sure that the reset function will be called at least once at initialization. You can find the reset function implementation in the **plugincore.cpp** file just below the initPluginParameters( ) function.

When you create your project, the function will have a default implementation shown here:

```
bool PluginCore::reset(ResetInfo& resetInfo)
{
// --- save for audio processing
audioProcDescriptor.sampleRate = resetInfo.sampleRate;
audioProcDescriptor.bitDepth = resetInfo.bitDepth;
// --- other reset inits
return PluginBase::reset(resetInfo);
}
//
```

The reset function argument is a **ResetInfo** structure reference. This simple structure has only two members, one for the sample rate and one for the bit-depth. The bit-depth information is not available in all APIs, and is somewhat of a legacy attribute, dating back to the fixed-point processing days and early plugin APIs, so we will ignore it in our plugins here. However, if you are writing a plugin that needs to know if the audio file feeding it is 16-bit versus 24-bit and the API allows it, then you may use this information accordingly. You can see from the default implementation that the sample rate is stored on our **audioProcDescriptor** member variable, and **you can access this value at any time and from any function in the core**.

After storing the sample rate, your plugin may need to perform other resetting operations including:

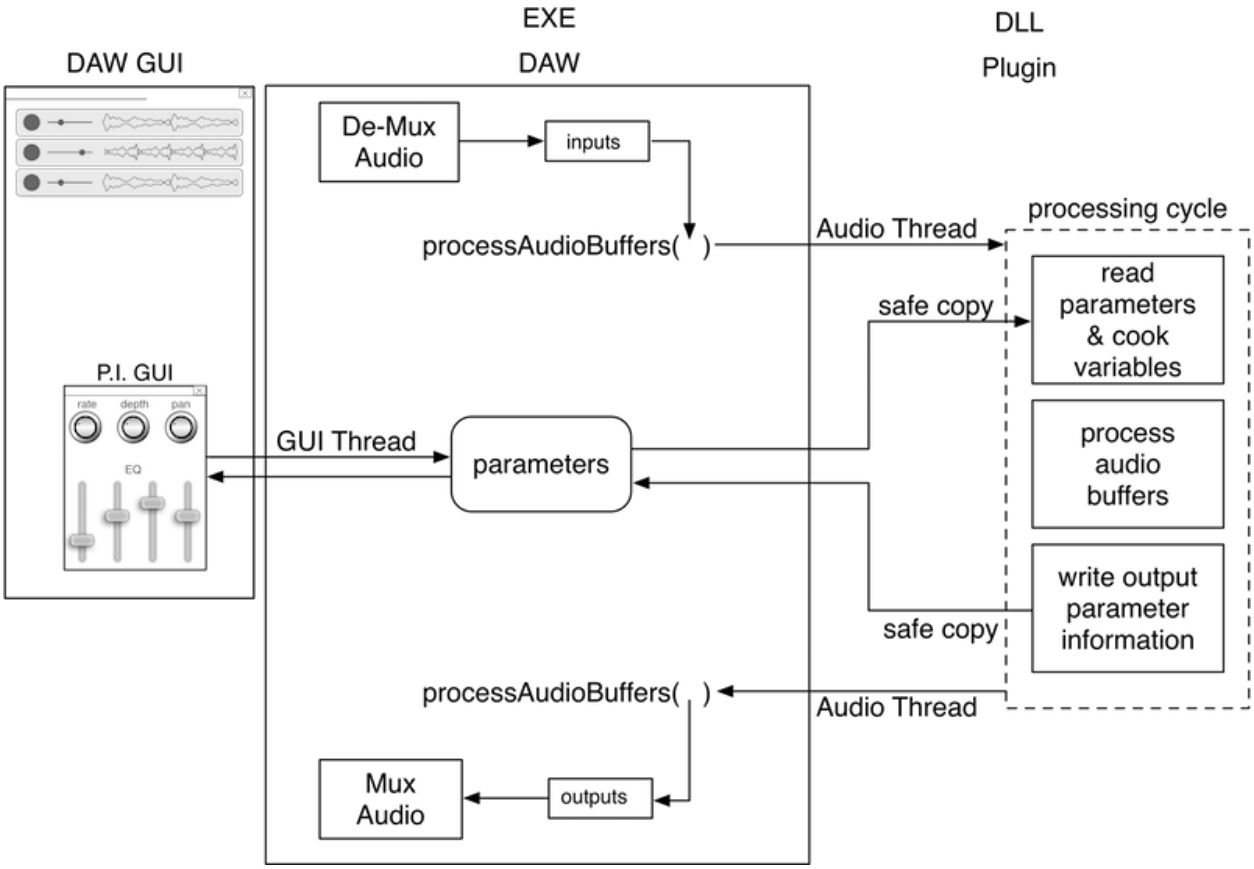
- re-calculating filter coefficients based on a new sample rate
- flushing delay registers and buffers of old audio data so it is not used during the next processing run
- re-sizing buffers that are sample-rate dependent (like the circular buffers we use for delay and reverb plugins)
- resetting pointers, index values or anything else to prepare for a fresh audio processing cycle

Nothing in our volume plugin's functionality is

going to require sample rate updates or resetting for fresh audio runs, so here we will not bother with it. However, in most of the plugin projects, there will be something to adjust based on the sample rate so we will include the reset function as part of the other audio processing function descriptions.

The Buffer Processing Cycle

The buffer processing cycle is complex but it is also defined and explained in high detail for AU, AAX, VST and the resulting ASPiK framework implementation in **Designing Audio Effects Plugins in C++**D 2nd Ed. by Will Pirkle so we won't go into major details here. The figure below shows the layout of a generic audio plugin's buffer processing cycle:



You can see from the above figure that the buffer processing cycle consists of three components: reading incoming parameter updates from the GUI controls, processing the audio, then writing output parameter information to the VU meters or other output-only controls.
The **PluginBase** defines three functions that correspond to this three-step architecture. These three functions will always be called in this exact order during each and every buffer processing cycle.

```
// --- preProcess: sync GUI parameters here
virtual bool preProcessAudioBuffers(ProcessBufferInfo& processInfo);
// --- process buffers - the DSP
virtual bool processAudioBuffers(ProcessBufferInfo& processBufferInfo);
// --- postProcess: do update meters, plots, graphs
virtual bool postProcessAudioBuffers(ProcessBufferInfo& processInfo);
//
```

Buffer Pre-Processing

During the pre-processing phase, we make a single call to the function that synchronizes our plugin variables with the values stored in our plugin parameters. During this phase, it is impossible for the DAW, GUI or other entity to manipulate the parameters or the variables. The pre-processing function is:

```
// --- do pre buffer processing
bool PluginCore::preProcessAudioBuffers(ProcessBufferInfo& processInfo)
{
// --- sync internal variables to GUI parameters
syncInBoundVariables();
return true;
}
//
```

The *syncInBoundVariables* method has two components. The first part simply copies the updated parameter information into the bound variables we supplied during the parameter instantiation. More importantly, this function calls a secondary function that allows you to do any post-update calculations. This base class function is implemented in the **PluginCore**:

```
bool postUpdatePluginParameter(int32_t controlID, double controlValue,
      ParameterUpdateInfo& paramInfo);
```

This function will be called for every parameter that is updated and will give you the chance to "cook" any variables that may need alteration as a result. We will demonstrate when you might use this function shortly. The incoming **ParameterUpdateInfo** structure contains boolean flags that will indicate why this function was called. Once the pre-processing phase is complete, your plugin needs to be fully setup to process audio. We will show an example of using this function later.

Buffer Post-Processing

The *postProcessAudioBuffers* method is called after all incoming buffers have been processed. The default implementation for this function is:

```
bool PluginCore::postProcessAudioBuffers(ProcessBufferInfo& processInfo)
{
// --- update outbound variables
updateOutBoundVariables();
return true;
}
```

This function simply calls the sub-function *updateOutboundVariables* which copies output parameter values into a special list that will be delivered back to the native parameter list, and then back to the GUI in a thread-safe manner. You won't need to modify or mess with that function, but this is the opportunity for your plugin to do any other processing after the buffer processing is completed. This would only include processing after ALL audio data has been processed.

-

Frame, Buffer and Sub-Block Processing

We have finally arrived at the function that will do all of the cool audio processing stuff our plugin was designed for. At this point, you need to make a fairly critical decision about how that processing will occur: by frame, buffer or sub-block.

The ASPiKreator software will setup the processing for you based on your selection from its interface. You may also edit the CMakeLists.txt file directly, modifying the statements in the **Plugin Options** section of the file, near the top. In this example, I am setting up processing sub-blocks of 64 samples per chanel. Note that if you set PROCESS_FRAMES TRUE, then that overrides all other statements.

```
# --- Plugin Options ---
set(PROCESS_FRAMES FALSE) # <-- set TRUE or FALSE
set(PROCESS_BUFFERS FALSE) # <-- set TRUE or FALSE
set(PROCESS_BLOCKS TRUE) # <-- set TRUE or FALSE
set(BLOCK_SIZE 64) # <-- numerical, in samples (per channel) if using blocks
###
```

DAW Buffers

All APIs deliver audio data to the buffer processing function in separated buffers, one for each channel. This is described in great detail in my FX plugin book. If the audio is stereo, then the pluign receives a pair of pointers, one for each buffer. The plugin uses these pointers to setup and execute the audio processing for FX, or audio rendering for software synthesizers.

Audio Frames

If we assemble one audio sample from each channel-buffer for each sample period, we create a **frame**. So a set of channel buffers containing **M** samples each would be broken into **M frames**. A stereo input frame would consist of an array that contains two audio samples, one for the left channel and one for the right. We might indicate that as {left, right}. A frame of surround sound 5.1 audio data is an array of six audio samples, one from each channel, organized in the following manner: {left, right, center, LFE, left surround, right surround} where LFE stands for **Low Frequency Effects** or the sub-woofer channel.

Frame procssing is inherently easier to understand, so people who are new to plugin programming often prefer it, at least in the beginning steps while learning. But, there are some algoriths that naturally lend themselves to frame processing directly. An example of this might be a stereo ping-pong delay plugin where left and right channel data are both needed on a sample-by-sample basis. Another example is a stereo-linked compressor, whose side-chain information is derived from both left and right inputs in a way such that the left channel cannot be processed without the right channel, and vice-versa. **Note that these may also be processed with buffers or sub-blocks, but you will need to modify the code to operate on multiple buffers or sub-block channels at once**.

When you process audio frames, you fill in the corresponding **PluginCore** function (details follow):

virtual bool processAudioFrame(ProcessFrameInfo& processFrameInfo)

Entire Buffers

The most efficient processing occurs when you process buffers as you are operating directly on the DAW buffers themselves, rather then smaller frame-based buffers that are setup for the frame processing function – these require an audio copying mechanism to move the data into and out of the DAW buffers.

- **Processing Independent Channels**: in this case you will loop over each channel-buffer and process them as un-related data; typically you use an outer for( ) loop on the channels, and an inner for( ) loop on the audio samples in each buffer
- **Processing Inter-connected Channels**: when your algorithm requres individual samples from each channel, you will usually use a for( ) loop

over the number of samples in the buffer, then use an algorthm that combines samples from each channel buffer in succession, processing them, then writing them to the output buffers in the same succession order

You have two choices for buffer processing - you may alter override and implement the base class function:

virtual bool processAudioBuffers(ProcessBufferInfo& processBufferInfo)

Or, you may simply use the sub-block processing function (described below) and treat each block as a complete buffer. You will still be operating directly on the DAW buffers, so this is very nearly as CPU efficient as buffer processing, only incurring a function call, that uses a const reference as an argument so there is no local copying (i.e. its fast). I use this method when I do buffer processing.

Sub-Blocks

When you process sub-blocks, you are operating on smaller pieces of the DAW buffers, but the operation is still direct with no audio sample copies being made. You define the size of these blocks, indicating the number of samples per channel in ech sub-block. The processing is the same for buffers, only your loops will be smaller. To help you with both Sub-Block and Buffer processing, the **PluginCore** comes pre-loaded with two functions, one for FX and the other for synths, that you may use as templates or starting points. When you process sub-blocks, the follwing **PluginCore** function is called; here is the code that is built-into each new plugin project:

```
// --- implement block processing
bool PluginCore::processAudioBlock(ProcessBlockInfo& processBlockInfo)
{
// --- FX or Synth Render
// call your block processing function here
// --- Synth
if (getPluginType() == kSynthPlugin)
renderSynthSilence(processBlockInfo);
// --- or FX
else if (getPluginType() == kFXPlugin)
renderFXPassThrough(processBlockInfo);
return true;
}
//
```

Here is the code for the two example helper functions.

- Synth: Notice how the MIDI events are processed at the top of the function, then the synth output is rendered. See my Synth book for more details on this paradigm, and **MIDI Message Processing** below
- FX: the input samples are simply copied to the output sample locations in the buffer; MIDI is sample accurate and MIDI messages will be inserted as the samples are processed (see **MIDI Message Processing**)

```
//
// --- Synth Silence
//
bool PluginCore::renderSynthSilence(ProcessBlockInfo& blockInfo)
{
// --- process all MIDI events in this block (same as SynthLab)
uint32_t midiEvents = blockInfo.getMidiEventCount();
for (uint32_t i = 0; i < midiEvents; i++)
{
// --- get the event
midiEvent event = *blockInfo.getMidiEvent(i);
// --- do something with it...
// myMIDIMessageHandler(event); // <-- you write this
}
// --- render a block of audio; here it is silence but in your synth
// it will likely be dependent on the MIDI processing you just did above
for (uint32_t sample = blockInfo.blockStartIndex, i = 0;
sample < blockInfo.blockStartIndex + blockInfo.blockSize;
sample++, i++)
{
// --- write outputs
for (uint32_t channel = 0; channel < blockInfo.numAudioOutChannels; channel++)
{
// --- silence (or, your synthesized block of samples)
blockInfo.outputs[channel][sample] = 0.0;
}
}
return true;
}
//
//
// --- FX Pass-Through
//
bool PluginCore::renderFXPassThrough(ProcessBlockInfo& blockInfo)
{
// --- block processing -- write to outputs
for (uint32_t sample = blockInfo.blockStartIndex, i = 0;
sample < blockInfo.blockStartIndex + blockInfo.blockSize;
sample++, i++)
{
// --- handles multiple channels, but up to you for bookkeeping
```

```
for (uint32_t channel = 0; channel < blockInfo.numAudioOutChannels; channel++)
{
// --- pass through code, or your processed FX version
blockInfo.outputs[channel][sample] = blockInfo.inputs[channel][sample];
}
}
return true;
}
//
//
```

Changing the Processing Paradigm

If you change your mind later or you want to experiment with the different processing types, you do NOT need to re-run CMake or use the ASPiKreator to make another project; instead you may edit the **plugindescription.h** file directly (this may only be done after the the project has been created with CMake). You will find the boolean and uint32_t variables that define the processing. You set them as follows:

Frame Processing

- Set kProcessFrames to true
- kBlockSize will be ignored

```
//
// --- Plugin Options
const bool kProcessFrames = true;
const uint32_t kBlockSize = DEFAULT_AUDIO_BLOCK_SIZE;
//
```

Buffer Processing

- Set kProcessFrames to false
- Set kBlockSize to WANT_WHOLE_BUFFER (which is a constant defined as 0)

```
//
// --- Plugin Options
const bool kProcessFrames = false;
const uint32_t kBlockSize = WANT_WHOLE_BUFFER;
//
```

Sub-Block Processing

- Set kProcessFrames to false
- Set kBlockSize to the value you want:

- the default size DEFAULT_AUDIO_BLOCK_SIZE, defined as 64 samples (there is a reason for this value that is tied to software synthesis and explained in my Synth book)
- another block size, e.g. 512 samples

To setup sub-block processing with the default buffer size:

```
//
// --- Plugin Options
const bool kProcessFrames = false;
const uint32_t kBlockSize = DEFAULT_AUDIO_BLOCK_SIZE;
//
```

To setup sub-block processing with 512 samples (as an example):

```
//
// --- Plugin Options
const bool kProcessFrames = false;
const uint32_t kBlockSize = 512;
//
```

-

MIDI Message Processing

MIDI messages are sent to the **PluginCore**'s buffer processing function as part of the normal **The Buffer Processing Cycle** and arrive in a queue, timestamped with the sample number corresponding to locations in the DAW I/O buffers. The **PluginBase** implements the **processAudioBuffers** method and fires MIDI events into the processing loop depending on your audio processing method.
The **midiEvent** structure holds the information for each MIDI message and has many member variables; here I am only showing the normal message members and the sample-accurate index value:

```
struct midiEvent
{
// --- constructors and overloaded = operators removed here
//
uint32_t midiMessage = 0;
uint32_t midiChannel = 0;
uint32_t midiData1 = 0;
uint32_t midiData2 = 0;
uint32_t midiSampleOffset = 0;
};
```

Regardless of the processing paradigm, the same MIDI function is usd to process these messages and uses a simple **midiEvent** structure to store the message data:

```
//
bool PluginCore::processMIDIEvent(midiEvent& event)
{
// --- IF PROCESSING AUDIO BLOCKS: push into vector for block processing
if (!pluginDescriptor.processFrames)
{
processBlockInfo.pushMidiEvent(event);
return true;
}
// --- IF PROCESSING AUDIO FRAMES: decode AND service this MIDI event here
// for sample accurate MIDI
// myMIDIMessageHandler(event); // <-- you write this
return true;
}
//
//
```

Notice how this function works:

Processing sub-blocks or buffers:

The **pluginDescriptor.processFrames** boolean will be **false**; in this case a special function is called to load the incoming MIDI message into a vector that will be usually be serviced, all at once, at the top of the block processing function; note that you may also perform this in a sample accurate manner, but it will require some coding on your part to match up the **midiEvent**'s midiSampleOffset index (the sample location within the DAW audio buffers) with the block or buffer (for buffer processing this is trivial as the sample offsets will match up with your for( ) loop), but for sub-block processing, you will need to keep track of the blocks and samples per buffer (see the preProcessAudioBuffers function for a location to set this up). However, there are good reasons for procssing the MIDI at the top of sub-blocks as detailed in my Synth book (and many synth manufacturers use this same paradigm).

```
// --- process all MIDI events in this block (same as SynthLab)
uint32_t midiEvents = blockInfo.getMidiEventCount();
for (uint32_t i = 0; i < midiEvents; i++)
{
// --- get the event
midiEvent event = *blockInfo.getMidiEvent(i);
// --- do something with it...
// myMIDIMessageHandler(event); // <-- you write this
}
// --- render a block of audio; here it is silence but in your synth
// it will likely be dependent on the MIDI processing you just did above
for (uint32_t sample = blockInfo.blockStartIndex, i = 0;
sample < blockInfo.blockStartIndex + blockInfo.blockSize;
sample++, i++)
{
// --- write outputs
for (uint32_t channel = 0; channel < blockInfo.numAudioOutChannels; channel++)
```

```
{
// --- silence (or, your synthesized block of samples)
blockInfo.outputs[channel][sample] = 0.0;
}
}
return true;
//
```

Processing Frames:
You insert your MIDI processing function (e.g. myMIDIMessageHandler( )) right inside of the processMIDIMessage( ) call here; the [PluginBase] will **interleave the MIDI message function calls with the processAudioFrames function calls** and your MIDI messaging is automatically sample accurate!

```
// --- IF PROCESSING AUDIO FRAMES: decode AND service this MIDI event here
// for sample accurate MIDI
// myMIDIMessageHandler(event); // <-- you write this
```

Processing the Audio Data

In the audio processing phase, we need to pick up the audio inputs, process them through our algorithm, and then write them to the audio outputs. **In this part of the programming guide, we will use frame processing exclusively.** You may easily extract and transfer this code into your block or buffer processing functions as you like; the reasons for focusing on frames here are:

1. frames are easy to think about - just one sample from each channel at a time
2. block processing ususally depends on some factor about the plugin's algorithms that may change from one plugin to the next
3. buffer (or block) procssing is just a for( ) loop that iterates over frames (samples from each channel) so the code is fundamentally the same.

The *processAudioFrame* function will handle the audio processing by frame. It's sole function argument is a data structure that contains all the information we need to process a buffer of audio. Note that this also includes information from the host about the DAW session including BPM and time-signature information. See [Frame, Buffer and Sub-Block Processing] for information you can use to convert this code to block or buffer procssing.

Information about the Frame

The **processAudioFrame** function might be the most important function we implement since it is where we will do the DSP magic for the plugin. If you want to process buffers, then you should still make note of this section, as some of the paradigms are similar. The frame processing function is shown here with its function parameter passed as a **ProcessFrameInfo** structure.

```
//
bool PluginCore::processAudioFrame(ProcessFrameInfo& processFrameInfo);
//
```

Let's examine that function argument's members:

```
struct ProcessFrameInfo
{
float* audioInputFrame = nullptr;
float* audioOutputFrame = nullptr;
float* auxAudioInputFrame = nullptr;
float* auxAudioOutputFrame = nullptr;// --- for future use
uint32_t numAudioInChannels = 0;
uint32_t numAudioOutChannels = 0;
uint32_t numAuxAudioInChannels = 0;
uint32_t numAuxAudioOutChannels = 0;
ChannelIOConfig channelIOConfig;
ChannelIOConfig auxChannelIOConfig;
uint32_t currentFrame = 0;
// --- for future use, VCVRack
float* controlSignalInputs = nullptr;
float* controlSignalOutputs = nullptr;
uint32_t numControlSignalInputs = 0;
uint32_t numControlSignalOutputs = 0;
HostInfo* hostInfo = nullptr;
IMidiEventQueue* midiEventQueue = nullptr;
};
//
```

At the top of the structure are the members that are the audio input and output pointers. Each frame of data consists of one sample from each channel packed into an array of float values. Float values are chosen because they are common across all APIs, thought VST3 does allow for processing doubles as well. The arrays are passed-by-pointer into the frame processing function. Just below these array pointer declarations are the unsigned integers (uint32_t) that tell you how many channels there are in a given frame array and these are easy to interpret: *numInputChannels* is the array size of the *audioInputFrame* array while *numOutputChannels* is the array size of the *audioOutputFrame* array. The audio samples are packed into the arrays according to the standardized audio file formats. These are published on the internet, and you can also find a very concise description of the channel packing formats in the vstspeaker.h file that is inside the VST3 SDK.

The *audioInputFrame* and *audioOutputFrame* arrays hold the frames for the audio input and output busses. The *auxAudioInputFrame* array holds audio data that is coming from a side-chain. We will

use this information in some of our side-chain-able plugins in later chapters.
The *auxAudioOutputFrame* is reserved for future use as plugin APIs begin to implement auxiliary data channels (none do it so far).

The ***ChannelIOConfig*** variables describe more information about the audio I/O configurations than we can with the channel counts alone. The **ChannelIOConfig** structure holds two member variables, one that describes the input and the other that describes the output channel configurations encoded as channelFormat enumerations. You can find the *channelFormat declaration* in the **pluginstructures.h** file. These are especially important for multi-channel plugins. For example, you will see that there are multiple channel formats for a given channel count; kCF7p1Sony, kCF7p1DTS, and kCF7p1Proximity all specify eight total channels (known as “7.1”) yet there are three different channel format specifications to deal with. In addition, your plugin may feature very different configurations from input to output; a fold-down plugin may accept a 7.1 input and deliver a plain stereo output.

Following the channel I/O information is the frame counter variable. This will indicate which frame of the buffer is being processed. We will pass this variable to the MIDI event queue firing mechanism. The **HostInfo** structure holds information from the host about the current buffer of data being processed, including the current session Beats Per Minute (BPM), the time signature numerator and denominator, and the audio track time at which the buffer occurs. There is even more information available such as musical bar position, but these are dependent on the API as well as the DAW that implements the specification; many of these host values are designated as optional and many DAWs ignore them. The last item in the array is a pointer to a MIDI event queue that holds MIDI events that were gathered at the same time as the audio buffer and time-stamped with index values that correspond to locations in the audio buffer. The entire MIDI queue-ing operation is handled in the ASPiK plugin shell code and you don't need to worry about it. As MIDI events are fired, your MIDI handlers will be called in sequence and in a thread-safe manner.

Audio Input and Output Samples

We will need to decode the **ChannelIOConfig** variable for the audio data and pick up the audio input data samples, process them, and then write our audio outputs into the *audioOutputFrame array* according to the output channel IO configuration. You may adapt this code as your plug requires it. The stock *processAudioFrame* function your plugin is created with looks something like this at the head of the function.

```
// --- fire any MIDI events for this sample interval
```

```
processFrameInfo.midiEventQueue->fireMidiEvents(processFrameInfo.currentFrame);
// --- do per-frame smoothing
doParameterSmoothing();
// --- Synth or FX Plugin?
if (getPluginType() == kFXPlugin)
{
// do FX stuff
}
//
```

From this point on, we will assume that we are processing for a FX plugin and we'll omit those enclosing curly brackets for brevity. All ASPiK plugins will by default accommodate mono-mono, mono-stereo and stereo-stereo formats. You can add more channel formats as you like. Each new blank ASPiK plugin will pass audio from input to output directly. You will then modify the code to do some meaningful processing. The audio data samples arrive in slots in the *audioInputFrame* and *audioOutputFrame* arrays and we use standard C++ array notation to access them. We might write something like this to pick up the input samples, process them through some interesting audio processing function then and pass the resulting samples to the outputs:

```
// --- get left and right input samples
double xn_L = processFrameInfo.audioInputFrame[0];
double xn_R = processFrameInfo.audioInputFrame[1];
// --- process them to create outputs
double yn_L = doSomeCoolAudioProcessing(xn_L);
double yn_R = doSomeCoolAudioProcessing(xn_R);
// --- send audio out
processFrameInfo.audioOutputFrame[0] = yn_L;
processFrameInfo.audioOutputFrame[1] = yn_R;
//
```

From this example, it should be obvious how to pick up input samples and write output samples. If we were processing a 5.1 input, we might write something like this instead:

```
// --- 5.1
double leftIn = processFrameInfo.audioInputFrame[0];
double rightIn = processFrameInfo.audioInputFrame[1];
double lfeIn = processFrameInfo.audioInputFrame[2];
double leftSurroundIn = processFrameInfo.audioInputFrame[3];
double rightSurroundIn= processFrameInfo.audioInputFrame[4];
//
```

To decode our three I/O combinations, we use the following code:

```
// --- Mono-In/Mono-Out
if(processFrameInfo.channelIOConfig.inputChannelFormat == kCFMono &&
processFrameInfo.channelIOConfig.outputChannelFormat = kCFMono)
{
// --- DO mono-in --> mono out processing
}
// --- Mono-In/Stereo-Out
```

```
if(processFrameInfo.channelIOConfig.inputChannelFormat == kCFMono &&
processFrameInfo.channelIOConfig.outputChannelFormat == kCFStereo)
{
// --- DO mono-in --> stereo out processing
}
// --- Stereo-In/Stereo-Out
if(processFrameInfo.channelIOConfig.inputChannelFormat == kCFStereo &&
processFrameInfo.channelIOConfig.outputChannelFormat == kCFStereo)
{
// --- DO stereo-in --> stereo out processing
}
//
```

You can modify the channel decoding logic to suit your own taste. One thing to note is that this function (like all of the **PluginBase** functions) returns a boolean variable to signify whether we actually processed data or not. We can return true from inside each of those curly bracket sets and return false if none of our channel combinations are supported.

-

The Audio Signal Processing

As the projects become more complex, we will typically create C++ objects to do the signal processing. This is good practice on many different levels. For example, suppose you created three really interesting and marketable plugins that did compression, EQ, and reverb and you later decided to consolidate them into one multi-function plugin. If the majority of your signal processing code were implemented in C++ objects, you would only need to aggregate these objects and chain their processing functions together. If you did not use C++ objects and instead had code scattered all over the frame processing function and various **PluginCore** functions, then consolidation would be difficult.

In our volume plugin here it does not make sense to create C++ objects to implement volume and mute, but we will use the C++ object paradigm as much as possible other example plugin projects. And you will find that the included **fxobjects.h** and **fxobjects.cpp** files are full of useful C++ audio processing objects that we will use throughout the book and you may use in any other C++ based signal processing system.

We can take advantage of the fact that we know that there must be a left channel present for any of our supported channel I/O combinations and simplify the code a bit and of course you are free to refactor the code however you like to make it more efficient or more readable. Looking at the volume plugin block diagram we see that the user adjusts the volume in dB. We need to convert the dB value of the GUI control to a raw multiplier for the audio signal processing using the following equation:

$$volume = 10^{\frac{volume_dB}{20}}$$

This introduces an important concept: the GUI control delivered a value that we could not immediately use. We needed to manipulate this value to arrive at something we can use directly in the signal processing routine. The data delivered from the GUI control is sometimes called **raw** data. We then take that data and cook it to create our final volume multiplier coefficient. This produces **cooked** data and the dB equation above can be thought of as the **cooking equation**. Very often the cooking process is complex, in which case we will create a separate **cooking function** to handle this chore. We will use the terms **raw** and **cooked** throughout the SDK when referring to these values. Referring to the block diagram again, we see that we need to not only calculate a volume multiplier value, but we also need to take into account the mute and channel settings. Rather than dwell on each line of code, let's examine a simple and perhaps naïve approach, disregarding the parameter smoothing or VST3 sample accurate automation options. Here we will create left and right channel volume multiplier values which will take on either the cooked volume value, or 0.0 if the channel selection does not include it, or if the mute switch is engaged. In this case, we may fill in the signal processing with this code:

```
// --- convert dB to raw
double volumeCooked = pow(10.0, volume_dB / 20.0);
double volume_L = volumeCooked;
double volume_R = volumeCooked;
// --- compare with channel setting
if (compareIntToEnum(channels, channelsEnum::stereo))
{
volume_L = volumeCooked; // redundant, just for demonstration
volume_R = volumeCooked;
}
else if (compareIntToEnum(channels, channelsEnum::left))
{
volume_L = volumeCooked;
volume_R = 0.0;
}
else if (compareIntToEnum(channels, channelsEnum::right))
{
volume_L = 0.0;
volume_R = volumeCooked;
}
// --- last in sequence = most significant
if (enableMute)
{
volume_L = 0.0;
volume_R = 0.0;
```

```
}
//
```

With our left and right channel volume multiplier values calculated we can then do the actual signal processing. Notice how the VU meter value is formed; since the **PluginParameter** will automatically rectify the signal as part of the meter ballistics operation, we only need to feed it the ordinary audio signal value of interest that is the output value here. Also, note how the mono-in-stereo-out logic simply copies the left output to both the left and right channels, which is appropriate for this plugin.

```
// --- left channel:
double xn_L = processFrameInfo.audioInputFrame[0];
double yn_L = volume_L * xn_L;
if(processFrameInfo.channelIOConfig.inputChannelFormat == kCFMono &&
processFrameInfo.channelIOConfig.outputChannelFormat == kCFMono)
{
// --- write to output
processFrameInfo.audioOutputFrame[0] = yn_L;
// --- set VU meter
vuMeter = yn_L;
return true; // processed
}
// --- Mono-In/Stereo-Out
if(processFrameInfo.channelIOConfig.inputChannelFormat == kCFMono &&
processFrameInfo.channelIOConfig.outputChannelFormat == kCFStereo)
{
// --- write to output
processFrameInfo.audioOutputFrame[0] = yn_L;
processFrameInfo.audioOutputFrame[1] = yn_L;
// --- set VU meter
vuMeter = yn_L;
return true; // processed
}
// --- Stereo-In/Stereo-Out
if(processFrameInfo.channelIOConfig.inputChannelFormat == kCFStereo &&
processFrameInfo.channelIOConfig.outputChannelFormat == kCFStereo)
{
// --- right channel:
double xn_R = processFrameInfo.audioInputFrame[1];
double yn_R = volume_R * xn_R;
// --- write to output
processFrameInfo.audioOutputFrame[0] = yn_L;
processFrameInfo.audioOutputFrame[1] = yn_R;
// --- sum L & R into mono
vuMeter = 0.5*yn_L + 0.5*yn_R;
return true; // processed
}
//
```

If you compile and test the plugin, you will see that it behaves as designed, though perhaps a bit underwhelming since it is just a volume control. Now let's step back and think about our CPU intensive calculation part: the only really complicated math involves cooking the raw dB value into a volume coefficient. But do we really need to run this equation on every function call to

processAudioFrame? This is going to depend on whether the volume parameter has undergone smoothing (either automatic, or via the VST3 sample accurate process).

-

Parameter Smoothing

ASPiK implements two types of parameter smoothing:

- native: you may setup the parameters to smooth themselves on each frame, buffer, or sub-block bounday (or any combination)
- VST3 Sample Accurate Automation (VST3SAA): for VST3 plugins only, Steinberg defines a method of smoothing parameters arriving from automation playback; if you use the native smoothing properly, you should not need VST3SAA however there may be some specialized plugin for VST3 only, in which you need it

Prior to ASPiK 1.6.8, the native and VST3SAA smoothing operations took place in the same function. For SDK 1.6.8 and beyond, these two smoothing operations have been separated out of a single function, and placed into two functions; you may enable/disable them independently (they have always been independent). Separating the two smoothing methods this way actually **saves CPU cycles** and parameters may only be smoothed once, with one version or the other. My advice is to implement native smoothing properly, then forget about VSTY3SAA smoothing and just leave it disabled (from the ASPiKreator or in **plugindescription.h**).

GUI Parameter Update/Cooking with Parameter Smoothing OFF
When parameter smoothing is turned off, the parameter's value will remain fixed and constant during the entire buffer processing cycle. This means that we are wasting CPU cycles by recalculating the cooked volume value on each audio frame. We really only need to calculate it once per buffer cycle and we can make use of the function **PluginCore::postUpdatePluginParameter** that will be called during the buffer pre-processing phase. Since this function is external to the *processAudioFrame* function, we will need to declare the cooked volume value as a member variable of the **PluginCore** object in the **plugincore.h** file:

```
// --- members
private:
```

```
// --- Continuous Plugin Variables
double volume_dB = 0.0;
// --- cooked volume value PluginCore::postUpdatePluginParameter()
double volumeCooked = 0.0;
// --- Discrete Plugin Variables
int enableMute = 0;
etc . . .
//
```

Then, in **PluginCore::postUpdatePluginParameter** we will decode the parameter controlID, and cook the incoming raw data. Because of our clever controlID enumeration naming convention, we don't need to look up the controlID, we simply use the enumeration which is the same as the variable name (controlID::volume_dB):

```
bool PluginCore::postUpdatePluginParameter(int32_t controlID, double controlValue,
      ParameterUpdateInfo& paramInfo)
{
// --- decode the controlID
switch(controlID)
{
case controlID::volume_dB:
{
// --- cooked variables
volumeCooked = pow(10.0, volume_dB / 20.0);
return true; // handled
}
default:
return false; // not handled
}
return false;
}
//
```

Now, we can remove the cooking calculation from the frame processing function (we'll just comment it out):

```
// --- convert dB to raw NOW DONE IN postUpdatePluginParameter
// double volumeCooked = pow(10.0, volume_dB / 20.0); <-- commented out!
double volume_L = volumeCooked;
double volume_R = volumeCooked;
//
```

ParameterUpdateInfo Options

There may be situations where you do not want to perform the post-update cooking function based on whether the parameter was changed during 1) the normal buffer cycle process, 2) a parameter smoothing operation, 3) a VST sample accurate update or 4) the user loading a preset. You might want to experiment with CPU usage rates or have other plugin-specific issues to deal with. For example, if the cooking process is CPU intensive because of some plugin option that the user has chosen, you might opt to disable the cooking in this situation. The third argument in the

postUpdatePluginParameter function is a **ParameterUpdateInfo** structure that delivers information about how and why the function is being called. You can examine the member variables and use them to alter your logic for the cooking process.

```
struct ParameterUpdateInfo
{
<SNIP SNIP SNIP>
bool isSmoothing;
bool isVSTSampleAccurateUpdate;
bool loadingPreset;
bool boundVariableUpdate;
bool bufferProcUpdate;
<SNIP SNIP SNIP>
};
```

These boolean flags are fairly straightforward to figure out by their names, but here the logic used.

- isSmoothing: ordinary parameter smoothing
- isVST3SampleAccurateUpdate: a VST3 sample accurate automation update
- loadingPreset: the user loaded a preset in the host DAW
- boundVariableUpdate: the variable binding operation has occurred
- bufferProcUpdate: updating at the top of the buffer process cycle

GUI Parameter Update/Cooking with Parameter Smoothing ON

When parameter smoothing is turned on, you have another easy option for cooking the GUI data. In the previous method using **PluginCore::postUpdatePluginParameter** you decoded each controlID and cooked each parameter separately. A simple alternative is to provide a function that cooks all incoming parameters together (at the same time). In the FX book examples, this function is named **updateParameters** but of course you may name it how you wish. This function's content will be highly dependent on the algorithm and GUI controls so I won't detail it here, but remember that it converts all GUI parameter values into meaninful data for your algorithm.

For example, when parameter smoothing or VST3SAA is enabled, the value of the *volume_dB* variable is going to be smoothed and updated on each sample interval at the top of the *processAudioFrame* function using either method:

1. **PluginCore::postUpdatePluginParameter**: here you cook each variable separately based on controlID

2. updateParameters: in this case you remove the code from postUpdatePluginParameter and place it into a single function that cooks all variables, mine is named updateParameters

Note that the ASPiK parameter smoothing is the **constant time** variety as opposed to constant rate. The smoothing operation always occurs over the interval that you set; typically 20-100mSec works fine, but for delay time controls, you should increase the parameter smoothing time to match the maximum delay time from the control, to provide a more analog sound as you move the control.

Parameter Smoothing with Frames

The processAudioFrame method already includes the call to the native and VST3SAA parameter smoothers (only one may be active at a time) via the **doParameterSmoothing** method. You can find the code for this near the top of the function. This is called once per sample interval, and then your cooking code will be called after each smoothing operation. The code here assumes you are using the updateParameters( ) paradigm:

```
//
bool PluginCore::processAudioFrame(ProcessFrameInfo& processFrameInfo)
{
// --- fire any MIDI events for this sample interval
processFrameInfo.midiEventQueue->fireMidiEvents(processFrameInfo.currentFrame);
// --- do per-frame smoothing
doParameterSmoothing(); // <-- this is the smoother
// --- IF you have your cooking code in one function, you MAY choose to call it
      here:
updateParameters( ): // <-- your cooking function that you name and write
//
```

Parameter Smoothing with Blocks

When processing blocks, you may perform the parameter smoothing and cooking functions ONCE per block, or interleaved with the block processing, at which point you have the same CPU loading as when processing frames. I prefer to process once per block as long as the blocks are small (64 samples or less). There is a special function for pre-processing prior to each block process, aptly named **preProcessAudioBlock** and this is where you may smooth and cook the parameters. This code includes a commented-out function call to your parameter cooking function.

```
bool PluginCore::preProcessAudioBlock(IMidiEventQueue* midiEventQueue)
{
// --- pre-process the block
processBlockInfo.clearMidiEvents();
// --- sample accurate parameter updates
for (uint32_t sample = processBlockInfo.blockStartIndex;
sample < processBlockInfo.blockStartIndex + processBlockInfo.blockSize;
sample++)
{
```

```
// --- the MIDI handler will load up the vector in processBlockInfo
if (midiEventQueue)
midiEventQueue->fireMidiEvents(sample);
}
// --- this will do parameter smoothing ONLY ONCE AT THE TOP OF THE BLOCK PROCESSING
//
// --- to perform per-sample parameter smoothing, move this line of code, AND your
      updating
// functions (see updateParameters( ) in comment below) into the for( ) loop above
// NOTE: smoothing only once per block usually SAVES CPU cycles
// smoothing once per sample period usually EATS CPU cycles, potentially
      unnecessarily
doParameterSmoothing();
// --- call your GUI update/cooking function here, now that smoothing has occurred
//
// NOTE:
// updateParameters is the name used in Will Pirkle's books for the GUI update
      function
// you may name it what you like - this is where GUI control values are cooked
// for the DSP algorithm at hand
// NOTE: updating (cooking) only once per block usually SAVES CPU cycles
// updating (cooking) once per sample period usually EATS CPU cycles, potentially
      unnecessarily
// updateParameters();
return true;
}
//
```

Parameter Smoothing with Buffers

Processing a buffer is the same concept as processing a very large block and you have the same options for breaking up the smoothing operation, all the way down to the sample (frame) level if desired. In this case, the processAudioBlock function will be based on entire buffers; or you may alternatively override and implement a **PluginCore::processAudioBuffers( )** function. In any event, it will be up to you to figure out how often you want to smooth and then re-cook the GUI parameters.

-

Information from the Host

Information is delivered to your plugin in the **HostInfo** structure that is passed into the processAudioFrame or processAudioBuffer argumen which includes DAW session data such as the current BPM and time signature. The information is valid for the duration of the frame or buffer.
A **HostInfo** pointer is delivered in both the **ProcessFrameInfo** and **ProcessBufferInfo** structures for the processAudioFrame and processAudioBuffer functions respectively. You may use the information included in it for any of the buffer process cycle functions as it will be guaranteed to be valid for the current buffer. Note that the amount and type of information available will depend on both the API (they vary significantly) as well as the DAW host (different hosts may support or not

support various information). However, the BPM and time signature information is universally supported and required of all APIs and hosts. An example of using this information to calculate eighth note delay times synchronized to the BPM in the processAudioFrame function might look like this:

```
//
if (processFrameInfo.hostInfo)
{
double quarterNoteDelay = 60.0 / processFrameInfo.hostInfo->dBPM;
double eighthNoteDelay = quarterNoteDelay / 2.0;
// etc...
}
//
```

You may also wish to play with the time signature as well. The numerator and denominator are found in:

```
//
processFrameInfo.hostInfo->fTimeSigNumerator
processFrameInfo.hostInfo->uTimeSigDenomintor
//
```

Notice that the numerator is a floating point value, so it may be fractional such as 3.5 while the denominator is an unsigned integer and will always be a whole number.

-

Adding more Channel I/O Formats

When you create a new ASPiK project, it will automatically declare these channel I/O combinations for you in the **PluginCore** constructor.

• Mono input to mono output (1 -> 1)
• Mono input to stereo output (1 -> 2)
• Stereo input to stereo output (2 -> 2)

The *processAudioFrame* function is already set up to decode these three I/O configurations as well. Adding more configurations is very easy and you can also add your own configuration definitions.

AU, AAX and VST all support different types of multi-channel operation that is beyond simple stereo. However, there are variations within each API, and there are some formats that are not supported in every API. So, if you decide you want to write a plugin for a multi-channel Ambisonics format you will likely not be able to write it for all APIs. For example, AU does not support channel I/O greater than

10 inputs and 10 outputs. On the other hand, VST3 supports up to 22.1 channel plugins, including variations for both Sony and DTS surround sound, while AAX supports Sony, DTS and most of the obscure Ambisonics formats. The first thing you need to decide is how many formats to support. Then, you need to describe each supported combination. If you support a side-chain input, you will need to describe the channel combinations your plugin will support (though there are some limitations on side-chains that are API-specific). This is best explained with the sample code, so go back to the **PluginCore** constructor. There are two base class functions that will make setting up your supported channels a simple operation. You just call the functions to add your supported combinations. The functions are:

```
addSupportedIOCombination( )
addSupportedAuxIOCombination( )
```

The channel configurations consist of a simple array that stores an input and output channel configuration pair. Enumerations for each type of channel configuration are found in the **pluginstructures.h** file.

```
//
<strong>enum channelFormat</strong><br>
{
kCFNone,
kCFMono,
kCFStereo,
kCFLCR,
kCFLCRS,
kCFQuad,
kCF5p0,
kCF5p1
etc . . .
}
```

You define a channel I/O configuration as an array pair using the enumerations above, and past this value to the helper functions. For ASPiK projects, this is pre-coded for you as:

```
// --- for FX plugins
if(getPluginType() == kFXPlugin)
{
addSupportedIOCombination({kCFMono, kCFMono});
addSupportedIOCombination({kCFMono, kCFStereo});
a ddSupportedIOCombination({kCFStereo, kCFStereo});
}
//
```

Besides mono and stereo, the most popular multi-channel format might be 5.1, which is encoded as *kCF5p1* above. To add support for 5.1 plugins, you would add the following to the above if() statement:

```
//
addSupportedIOCombination({kCF5p1, kCF5p1});
//
```

For a 5.1-to-stereo fold-down plugin, you would write:

```
//
addSupportedIOCombination({kCF5p1, kCFStereo});
//
```

You can also see that there are both Sony and DTS versions of higher channel-count formats. After describing your plugin's normal input/output channel configurations, you do the same for the side-chain using the aux IO variables. If you enable the side chain during the ASPiK CMake setup, then this code is written for you.

```
// --- for sidechaining, we support mono and stereo inputs; <br>
addSupportedAuxIOCombination({kCFMono, kCFNone});
addSupportedAuxIOCombination({kCFStereo, kCFNone});
//
```

-

Adding Factory Presets

Adding factory presets to your ASPiK plugin project is fairly straightforward. The **PluginCore** object contains a function named **PluginCore::initPluginPresets( )** and you will need to create **PresetInfo** objects for each preset. This is easiest to explain with a few examples. You will need to:

1. create a new **PresetInfo** object and give it a zero-based index value and a name string
2. call the helper function **initPresetParameters( )** to setup the object
3. call **setPresetParameter( )** for each **PluginParameter** object in your plugin, along with the control ID and value
4. call the helper function **addPreset( )** to add the object to the plugin's list

Here is a code example that will show you how it works - you only need to set the actual (not normalized) value for each parameter in the plugin:

```
//
bool PluginCore::initPluginPresets()
{
// --- Plugin Presets
int index = 0;
PresetInfo* preset = nullptr;
// --- Preset: Factory Preset
preset = new PresetInfo(index++, "Factory Preset");
initPresetParameters(preset->presetParameters);
setPresetParameter(preset->presetParameters, controlID::volume_dB, 0.000000);
setPresetParameter(preset->presetParameters, controlID::enableMute, -0.000000);
setPresetParameter(preset->presetParameters, controlID::channels, -0.000000);
addPreset(preset);
// --- Preset: MinusTwelve
preset = new PresetInfo(index++, "MinusTwelveLeft");
initPresetParameters(preset->presetParameters);
setPresetParameter(preset->presetParameters, controlID::volume_dB, -12.000000);
setPresetParameter(preset->presetParameters, controlID::enableMute, -0.000000);
setPresetParameter(preset->presetParameters, controlID::channels, 1.000);
addPreset(preset);
return true;
}
//
```

If your plugin has a large number of parameters, you can use a built-in ASPiK exclusive to add a button to your GUI that will write a .txt file that contains the above code for one complete preset. You can then copy and paste that code right into your project. See the Plugin GUI section for more information on using this helper control.

•

The PluginGUI Object

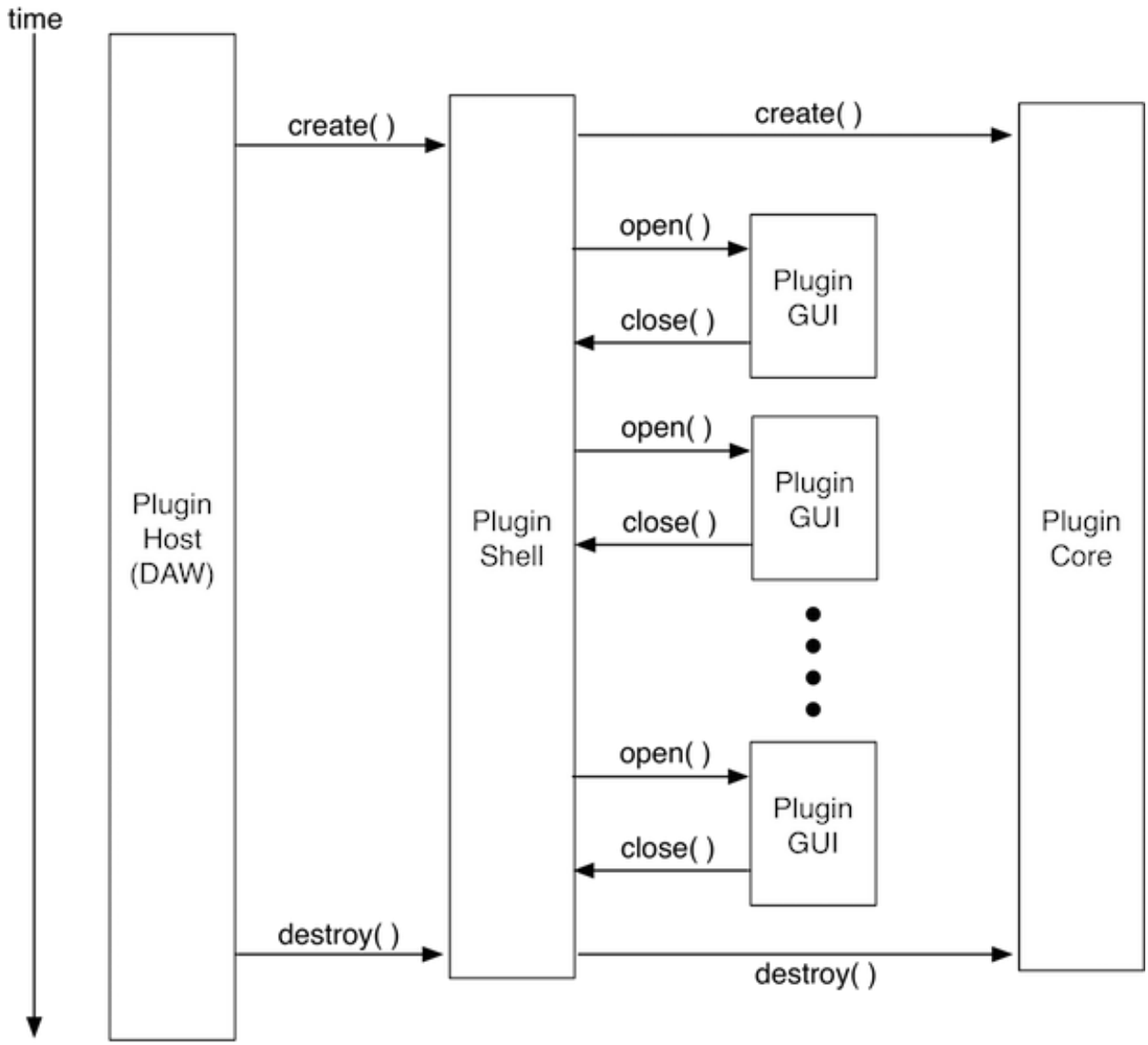
PluginGUI

The PluginGUI object handles all of the GUI chores for the Plugin Kernel. This includes:

• Generating the GUI graphically
• Initializing the GUI controls
• Transmitting control changes from the GUI to the native plugin shell in order to affect parameter changes
• Transmitting automation gesture messages to the native plugin shell and host DAW
• Receiving control updates from project saved-states
• Receiving control updates from running automation
• Creating the drag-and-drop GUI Designer for you to use in the graphical creation of your plugin GUI

It is important to remember that the PluginGUI and **PluginCore** objects are completely independent

of one another and do not need each other's existence to operate nor do they know about each other's existence. The Native plugin shell creates the GUI and core objects. This figure shows the lifecycles of the plugin host DAW, the native plugin shell, the **PluginCore** and PluginGUI objects. The user requests a plugin instance from the host, which creates the API-specific native plugin shell object. The shell creates the **PluginCore** object as part of its construction. The **PluginCore** exists as long as the shell object does and is destroyed in the shell's destructor. However, the user may open and close the GUI as many times as they wish during the plugin's lifetime. If the GUI is open at the time the user destroys the plugin, it will be closed first, then destroyed.



VSTGUI4

GUI Design with VSTGUI4

There are several GUI design packages available for use in C++ software. Steinberg developed the VSTGUI GUI generation package around the same time as their first plugin API named VST and implemented platform-independent GUI controls packaged in C++ objects. This software has been refined and updated to its current fourth generation state known as VSTGUI4. We will refer to it simply as VSTGUI. This GUI design software was developed with audio plugins in mind right from the beginning. It has matured into a quite remarkable open-source library of GUI objects that may be used for any plugin API. VSTGUI is ideal for our Plugin Kernel because:

• It is not plugin API-specific (as is the Plugin Kernel)
• It is platform independent (like the Plugin Kernel)
• It is a mature, fourth generation product
• It is well maintained and updated with new features and bug fixes
• Its designers stay on top of the latest C++ developments and extensions, incorporating them into each new major revision
• It is designed to be extendible with Custom-View and Sub-Controller objects and APIs
• It is deep – you can really go far down the rabbit hole with VSTGUI and it can be used to develop UIs for all kinds of software, not just audio plugins; for example the main UI in RackAFX is done entirely with VSTGUI
• The complete GUI description, including the graphics file data, may be written into a single XML file, making it ultra-portable
• It comes pre-packaged with a platform independent drag-and-drop GUI designer, greatly simplifying the process of GUI design

That last point is one of the supreme advantages of VSTGUI, enabling you to design a fully functional and professional GUI without writing a single line of code.

The Plugin Kernel makes use of this functionality so that you may design your GUI in the plugin API and platform of your choice. Since your entire GUI is described in one XML file, you can easily move your GUI from one platform and API to another, without needing to import graphics files, change a single line of XML code, or do any additional programming.

Modifier Keys & GUI Scaling

Modifier Keys

The PluginGUI object is already setup to implement the various modifier key and mouse click combinations used to fine tune the GUI and reset its controls to their default values. Modifier keys include the command (Mac), control, and alt (or option) keys. Pro-Tools defines its own API-specific set of modifier. This code is already written for you so there is nothing to implement however you will need to document these modifier combinations for your users. The modifier keys are listed below. Each involves clicking on a control with the left mouse button while holding a modifier key down.

Fine Adjustment: hold modifier key while clicking / dragging mouse over knob or slider
Pro-Tools MacOS: CMD
Pro-Tools Windows: CTRL
All others (Mac & Win): SHIFT

Go to Default Value: hold modifier key while clicking mouse over knob, slider, button
Pro-Tools MacOS: OPTION
Pro-Tools Windows: ALT
AU: OPTION
VST MacOS: CMD
VST Windows: CTRL
RackAFX: CTRL

Pro-Tools: Enable Automation CTRL + ALT + CMD while clicking on a GUI control

GUI Scaling

VSTGUI has a built-in GUI scaling mechanism that allows you to easily expand or shrink the entire GUI window. Sometimes this is required for high-resolution displays (shrinking) or for visually impaired users (expanding). There is a more accurate method for dealing with the Hi-DPI displays that you can also use (see the ASPiK website for more information). However, we can use the built-in scaling to made moderate adjustments to the GUI size. There is already a preset GUI control reserved for this operation and your PluginGUI object implements the GUI scaling, which VSTGUI calls “zooming.” The PluginGUI object can scale down to 65% or up to 150% of the normal GUI size. Values outside this range generally do not give acceptable results, however as with everything else, all the code is there for you to see and modify if you wish. We have already set up a GUI control that you can implement as a drop-list control to allow the user to scale the GUI. To add this control, add the following parameter instantiation after the rest of your GUI parameter declarations in

the **plugincore.cpp** file. Notice the special control ID value "SCALE_GUI_SIZE" here:

```
// — SCALE_GUI_SIZE
piParam = new PluginParameter(SCALE_GUI_SIZE, "Scale GUI",
"tiny,small,medium,normal,large,giant", "normal");
piParam->setIsDiscreteSwitch(true);
addPluginParameter(piParam);
```

There are a few more reserved ID values:

RESERVED PARAMETER ID VALUES
const unsigned int PLUGIN_SIDE_BYPASS = 131072; // VST3 only
const unsigned int XY_TRACKPAD = 131073;
const unsigned int VECTOR_JOYSTICK = 131074;
const unsigned int PRESET_NAME = 131075;
const unsigned int WRITE_PRESET_FILE = 131076;
const unsigned int SCALE_GUI_SIZE = 131077;

You can also see the reserved control ID value named WRITE_PRESET_FILE; this is for a helper control to let you easily assemble your presets. We will discuss that helper control later in the tutorial.

-

• VSTGUI4 Objects

- The VSTGUI library is chock full of GUI control objects. These include the most common plugin GUI controls:

  • Knobs
  • Sliders
  • Buttons and switches
  • Text labels
  • Text edit controls
  • Menus and drop-down displays
  • VU meters
  • Track-pads (aka XY-pads)
  • Background images

  In addition, there are many other types of controls and views, including:

• View containers (for aggregating sets of controls or other views)
• Splitter Views with drag-able pane-splitters
• Table views (rows and columns)
• Animation Views (for embedding animated content)
• A piano keyboard control
• Custom views for displaying plugin information (e.g. waveforms, graphs, etc…)

All of the VSTGUI library control objects that may be visible on the GUI are derived from the mother-of-all views called **CView**. This object primarily defines functions for drawing to the screen and handling mouse click events. Any library object that is capable of transmitting or receiving a plugin parameter value of some kind is derived from **CControl** which defines functions for getting and setting values as well as establishing a link to a specific parameter via a control-tag, or what we call a controlID.

•

CControl is derived from CView so it inherits the view's drawing and mouse event functions. One interesting example is the **CTextLabel** object. Ordinarily, you use these labels as static text controls – for example, you might place a text label "Filter Type" above a control that allows the user to select a filter, and "Filter Q" below the knob that adjusts the filter Q. In these cases, there are no links to any underlying plugin parameters – there is only static text to display. However, you might decide to place text on the UI, which is constantly updated and changing, such as a counter or other timing display. In this case, you would need to link the text control to an underlying plugin parameter's controlID. Therefore, the CTextLabel is derived from CControl rather than just CView.

Although there is absolutely zero C++ code for you to write in order to generate your plugin GUI, there are some concepts and details about the various VSTGUI objects that you need to know in order to make the GUI design process as simple as possible. There are some common design aspects to all of the VSTGUI controls that you should understand.

• The control's size is specified in screen pixels (sometimes called logical units)
• The control's location on the GUI is specified as an x-y coordinate pair that designates the top and left coordinates of the rectangle that defines the object's dimension; these coordinates are automatically set for you when you use the drag-and-drop GUI designer
• Most controls are linked to an underlying plugin parameter via the parameter's controlID value which is called a control-tag or simply tag in VSTGUI parlance
• Most controls require a graphics file (always packaged as a Portable Network Graphics or PNG file) to render the final control object; if you forget to assign a graphics file, then the control will appear to be invisible, though clicking on the invisible control will usually still

work properly. A few controls require two graphics files, for example the VU meter object requires a graphic for the ON state and another for the OFF state.

Here is a breakdown of the GUI controls we will be using along with their requirements or other information related to their creation. We will discuss some techniques to simplify the GUI design even further a bit later, allowing you to create GUIs very rapidly. These include auto-sizing and GUI templates. The VSTGUI objects presented here can be divided into two groups: controls that specifically require one or more graphics (PNG) files and those for which graphics are optional or not used.

Objects that require graphics files:
• CAnimKnob
• CKickButton
• COnOffButton
• CVuMeter

Objects that do not require graphics files:
• COptionMenu
• CSegmentButton
• CTextButton
• CTextLabel
• CTextEdit